

# 7주차 · 웨이포인트 유도 — atan2 와 LOS

캡스톤디자인 2026-2 · 충남대학교 자율운항시스템공학과 | 갱신 2026-09-03

## ★ 참조 강의 — 본 과목이 전제하는 배경

아래 다섯 과목은 본 과목 담당 교수가 직접 강의한 것이며, 본 과목이 전제하는 배경 지식에 해당함. 학부 기초에서 대학원 과정까지 이어지므로 부족한 지점부터 시작하면 됨. 본 문서에서 쓰는 좌표계·기호·유도 과정은 아래 강의에서 상세히 다루므로, 선수 지식이 부족한 경우 먼저 보고 돌아올 것.

| # | 과목                                            | 수준  | 언어  | 영상   | 자료·코드 |
|---|-----------------------------------------------|-----|-----|------|-------|
| 1 | 제어공학 — 전달함수, 되먹임, 안정도, 근궤적, PID. 모든 것의 토대     | 학부  | 한국어 | 재생목록 | 드라이브  |
| 2 | 제어시스템설계 — 해석이 아니라 설계. 사양 결정, 루프 정형화, 이산 구현    | 학부  | 한국어 | 재생목록 | 드라이브  |
| 3 | 캡스톤디자인 — 본 과목의 지난 학기 강의. 이동체 프로젝트를 처음부터 끝까지   | 학부  | 한국어 | 재생목록 | 드라이브  |
| 4 | 제어공학특론 — 좌표계, 6자유도 운동방정식, 회전행렬과 오일러각, 선형화와 트림 | 대학원 | 영어  | 재생목록 | 드라이브  |
| 5 | 센서신호처리 및 융합 — 센서 모델, 잡음, 추정, 다중센서 융합          | 대학원 | 영어  | 재생목록 | 드라이브  |

**MATLAB-Simulink** 가 처음이라면 **6주차 실습 전에 아래를 끝낼 것**. 본 과목의 제어가 실습은 전부 Simulink 로 진행함.

Onramp 는 무료이며 각각 몇 시간이면 끝남

| 도구       | 시작 지점                                                       |
|----------|-------------------------------------------------------------|
| MATLAB   | <a href="#">MATLAB Onramp · Core MATLAB Skills</a>          |
| Simulink | <a href="#">Simulink Onramp</a> · 담당 교수 Simulink 강의 1부 · 2부 |

- **과목:** 캡스톤디자인 (2026-2) · 충남대학교 자율운항시스템공학과
- **이번 주차 학습 내용:** 웨이포인트를 주고 배가 경로를 따라가게 만들. 유도법칙 두 가지를 비교함

## ★ 시작 전 확인

- **강의자료부터 갱신:** VS Code WSL 창 터미널에서 `cd ~/Capstone-Design && git pull` — 문서와 코드·모델이 같은 판이 됨 ([강의자료는-한-번-받고-git-pull-로-갱신한다](#), 처음 받는 법은 2주차 2-7)
- 6주차 모델( `W06_4_inner_loop.slx` )이 동작했어야 함
- MATLAB **R2024b** + **Simulink** + **ROS Toolbox**
- 배포 폴더 `W07_simulink/` 와 `W07_matlab/`

## i 6주차는 제어만 했음

- 6주차: "선수각을 45도로 맞춰라", "속도를 1.5 m/s 로 맞춰라"
- 그런데 누가 45도를 정해 주는가?
- 그것을 정하는 것이 **유도(Guidance)** 이며, 이번 주차의 학습 대상임

## 이 주차를 마치면 할 수 있어야 하는 것

1. 유도 · 항법 · 제어를 구분해서 설명하기
2. 선수각  $\psi$  · 침로각  $\chi$  · 크랩각  $\beta$  의 차이 설명하기
3. **cross-track error** 를 계산하기
4. **atan2** 유도 와 **LOS** 유도를 각각 구현하고 차이를 수치로 제시하기
5. lookahead distance  $\Delta$  가 무엇을 바꾸는지 실험으로 보이기
6. 조류가 있을 때 두 유도법칙이 어떻게 달라지는지 설명하기
7. 같은 제어기를 **오프라인 모델**과 **Gazebo** 양쪽에서 돌리기

## 준비물

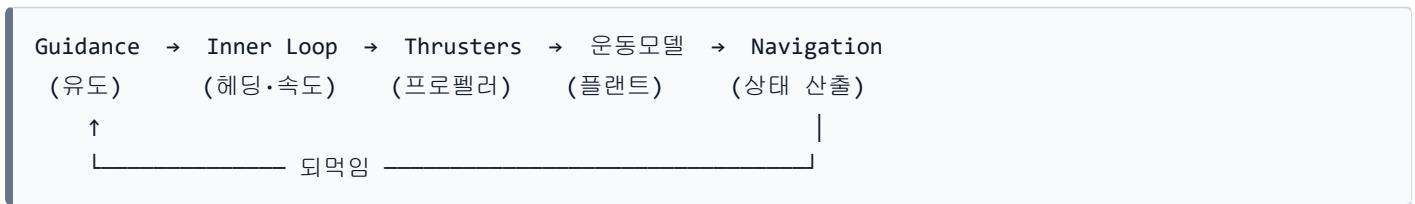
| 항목     | 내용                                                                                                                                                                                                                  |
|--------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 환경     | 1~6주차 WSL2 + ROS 2 + VRX                                                                                                                                                                                            |
| MATLAB | R2024b + Simulink + ROS Toolbox                                                                                                                                                                                     |
| 배포 파일  | <a href="#">W07_simulink/</a> (모델 2개 + 스크립트 10개: setup · plot · animate · compare · sweep · lookahead_demo · crab_demo · vrx_run · build · tidy_layout), <a href="#">W07_matlab/</a> (연구실 참고 코드 — 목록은 문서 끝 "배포 파일") |

★ 자율운항은 세 덩어리로 나뉨

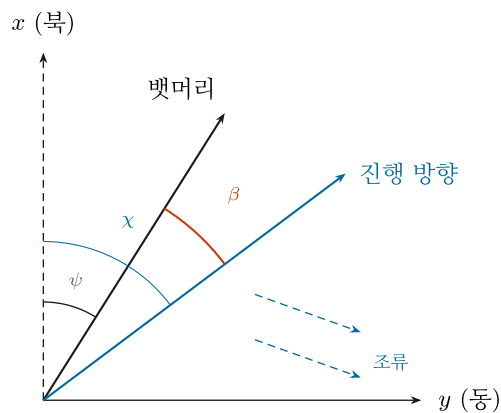
★ 자율운항은 세 덩어리로 나뉨

| 이름              | 답하는 질문               | 본 과목에서                    |
|-----------------|----------------------|---------------------------|
| 항법 (Navigation) | 나는 지금 어디에 있는가        | 6주차 ground truth odometry |
| 유도 (Guidance)   | 어디로 향해야 하는가          | 이번 주차                     |
| 제어 (Control)    | 그러려면 무엇을 얼마나 밀어야 하는가 | 6주차 헤딩·속도 제어              |

- 영문 머리글자를 따서 **GNC** 라고 부름
- 이번 주차에 작성할 모델도 이 순서 그대로 왼쪽에서 오른쪽으로 배치됨



## 1-2. 뱃머리가 향한 곳과 배가 가는 곳은 다르다



$\psi$  = 선수각 — 뱃머리가 향한 방향

 $\chi =$  침로각 — 배가 실제로 가는 방향 $\beta = \text{크랩각} - \text{둘의 차이}$ 

$\psi \cdot \chi$  는 북에서,  $\beta$  는 뱃머리에서

모두 시계방향으로 잔다

$$\chi = \psi + \beta \qquad \beta = \text{atan2}(v, u)$$

$$\beta = \text{atan2}(v, u)$$

 $u, v =$  선체 축으로 잰 전후 · 좌우 대지속도

선수각  $\psi$  만 제어하면 배는 경로에서 옆으로 밀린 채 나아간다. 조류에서 경로 이탈이 남는 이유다

## 용어 정리

| 기호             | 이름               | 뜻                                   |
|----------------|------------------|-------------------------------------|
| $\psi$ (psi)   | 선수각 (heading)    | 뱃머리가 향한 방향                          |
| $\chi$ (chi)   | 침로각 (course)     | 배가 실제로 가는 방향                        |
| $\beta$ (beta) | 크랩각 (crab angle) | 둘의 차이. $\beta = \text{atan2}(v, u)$ |

$$\chi = \psi + \beta, \quad \beta = \text{atan2}(v, u)$$

- $u$  = 대지(ground) 기준 속도의 선체 전후 성분,  $v$  = 같은 속도의 선체 좌우 성분

- 물에 대한 속도가 아니라 땅에 대한 속도임. 그래야  $\chi = \psi + \beta$  가 실제 진행 방향이 됨
- 조류나 바람이 없고 옆으로 미끄러지지 않으면  $v \approx 0 \Rightarrow \beta \approx 0 \Rightarrow \chi \approx \psi$
- 조류가 옆에서 밀면  $v \neq 0 \rightarrow$  뱃머리는 앞을 보는데 배는 옆으로 흘러감

#### i 게가 옆으로 걷는 모습에서 온 이름임

비행기에서도 같은 일이 일어남. 옆바람 착륙에서 기수를 바람 쪽으로 틀고 활주로를 따라 내려오는 것을 crab 이라고 부름

### 왜 지금 이 얘기를 하는가

- 이번 주차에 작성할 제어기는 선수각  $\psi$  를 제어함
- 그런데 경로를 따라가려면 침로각  $\chi$  가 맞아야 함
- 조류가 있으면  $\psi$  를 아무리 잘 맞춰도 경로에서 옆으로 밀린 채 나아감
- 이 사실이 이번 주차 실험의 핵심임

## 1-3. 가장 단순한 유도 — atan2

- 다음 웨이포인트를 곧장 겨냥함

$$\psi_{\text{ref}} = \text{atan2}(y_{wp} - y, x_{wp} - x)$$

- 식 한 줄로 구현됨. 연구실 파이썬 코드( `wamv_pid_control_v2.py` )도 이 방식임
- 웨이포인트에 충분히 다가가면 다음 점으로 넘어감. 그 판정을 어떻게 하는지는 §1-8 에서 다룸

#### atan2 를 쓰는 이유

`atan( $\Delta y / \Delta x$ )` 은  $-90^\circ \sim +90^\circ$  만 나옴. 뒤쪽으로 가야 할 때를 표현하지 못함  
`atan2` 는 두 인자의 부호를 함께 보고  $-180^\circ \sim +180^\circ$  전체를 줌

## 1-4. 그런데 그것으로 부족하다

### ⚠ atan2 유도는 경로 정보를 사용하지 않음

아는 것은 "다음 점이 저기 있다" 뿐임. 지금 경로에서 얼마나 벗어났는지 모름

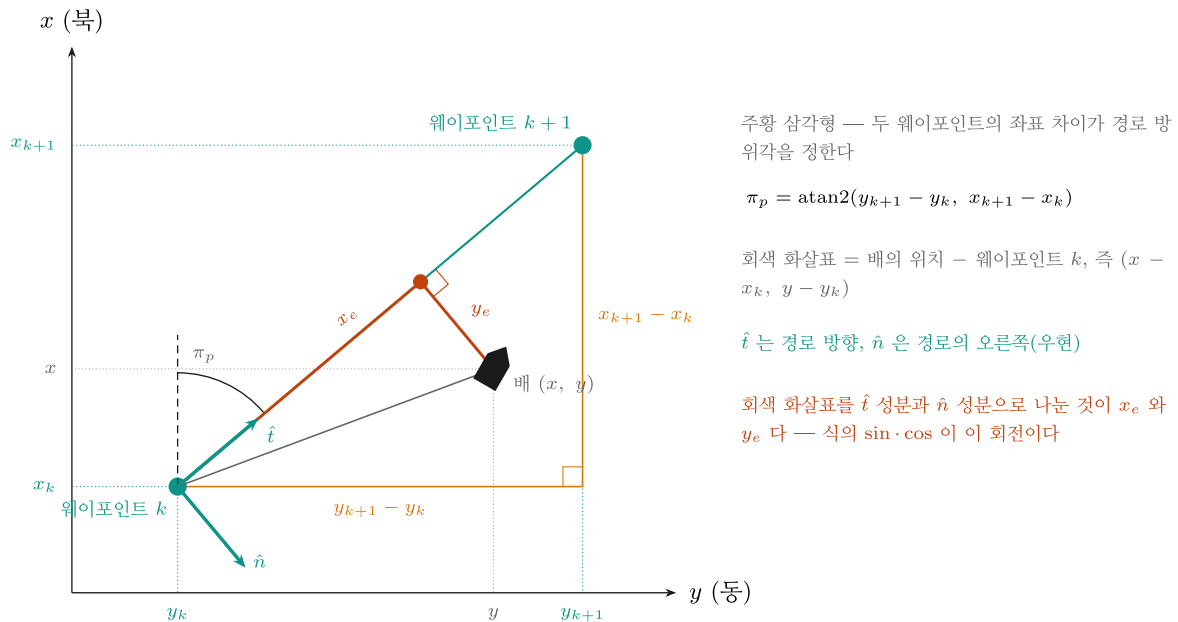
- 배가 경로 옆으로 벗어나 있어도, 목표점 방향만 맞으면 그대로 감
- 결과: 비스듬히 가로질러 목표점에 도착함
- 목표점에는 닿지만 지나온 자리가 계획한 선이 아님

### 왜 문제인가

| 상황                    | 경로를 벗어나면       |
|-----------------------|----------------|
| 장애물 필드 통과 (12주차 충돌회피) | 계획한 안전 통로를 벗어나 |
| 좁은 수로                 | 좌초             |
| 도킹 접근 (14주차)          | 접근각이 틀어져 접안 실패 |

## 1-5. 두 가지 오차 — 경로 방향과 경로 이탈

- 경로를 따라간다는 말을 숫자로 바꾸려면 오차가 두 개 필요함. 하나로 부족함
- 경로는 자기 좌표계를 만들. 그 좌표계로 배의 위치를 옮겨 적으면 두 오차가 한꺼번에 나옴



### 먼저 경로 방위각

- 웨이포인트  $k$  에서  $k+1$  로 가는 직선이 향한 방향. 북에서 잼

$$\pi_p = \text{atan2}(y_{k+1} - y_k, x_{k+1} - x_k)$$

- 한 구간 안에서는 상수임. 웨이포인트가 바뀔 때만 다시 계산함
- `atan2` 를 쓰는 이유는 §1-3 과 같음. 구간이 네 방향 중 어디로 가든 구분해야 함

### 경로가 만드는 두 방향

- $\hat{t}$  는 경로를 따라가는 방향,  $\hat{n}$  은 경로의 오른쪽(우현) 임

$$\hat{t} = \begin{bmatrix} \cos \pi_p \\ \sin \pi_p \end{bmatrix}, \quad \hat{n} = \begin{bmatrix} -\sin \pi_p \\ \cos \pi_p \end{bmatrix}$$

- $\hat{n}$  은  $\hat{t}$  를 시계방향(북→동)으로  $90^\circ$  돌린 것임. NED 의  $x$ (북)- $y$ (동) 평면에서 그 방향이 우현임
- 두 방향은 서로 수직이고 길이가 1 이므로, 어떤 벡터든 이 둘로 한 가지 방법으로만 쪼개짐

### 두 오차

- 웨이포인트  $k$  에서 배까지의 벡터를  $\hat{t}$  성분과  $\hat{n}$  성분으로 나눈 것이 두 오차임

$$x_e = (x - x_k) \cos \pi_p + (y - y_k) \sin \pi_p$$

$$y_e = -(x - x_k) \sin \pi_p + (y - y_k) \cos \pi_p$$

| 기호      | 뜻                                                                                           | 쓰는 곳               |
|---------|---------------------------------------------------------------------------------------------|--------------------|
| $x_e$   | 경로 방향 오차 — 구간을 얼마나 왔는가                                                                      | 웨이포인트 전환 판정 (§1-8) |
| $y_e$   | 경로 이탈량 — 경로에서 얼마나 옆으로 벗어났는가                                                                 | 유도법칙 (§1-6)        |
| $x, y$  | 배의 북쪽 · 동쪽 위치 (NED). $x_k, y_k$ 는 웨이포인트 $k$ 의 위치. 첨자 $e$ 가 붙은 $x_e, y_e$ 와 구분함. $N$ 은 요 모멘트 | 둘 다                |
| $\pi_p$ | 경로 방위각                                                                                      | 둘 다                |

- 두 식의 **sin** 과 **cos** 은 **회전 하나**임. 외울 것이 아니라 그림에서 읽는 것임
- $d$  를 구간 길이라 하면  $d - x_e$  가 **남은 거리**임. 이 값이 §1-8 의 전환 판정에 그대로 들어감

### ! 부호 규약

$y_e > 0$  이면 경로의 **오른쪽(우현)**,  $y_e < 0$  이면 **왼쪽**임

NED 에서 진행 방향을 바라볼 때 오른쪽이 양수임. 6주차의 요모멘트 부호와 같은 규약임  
그래서 우현으로 벗어나면 **좌현으로** 꺾어야 하고, §1-6 의 식이 빠기인 이유가 이것임

### ⚠ 확인용 값 세 개

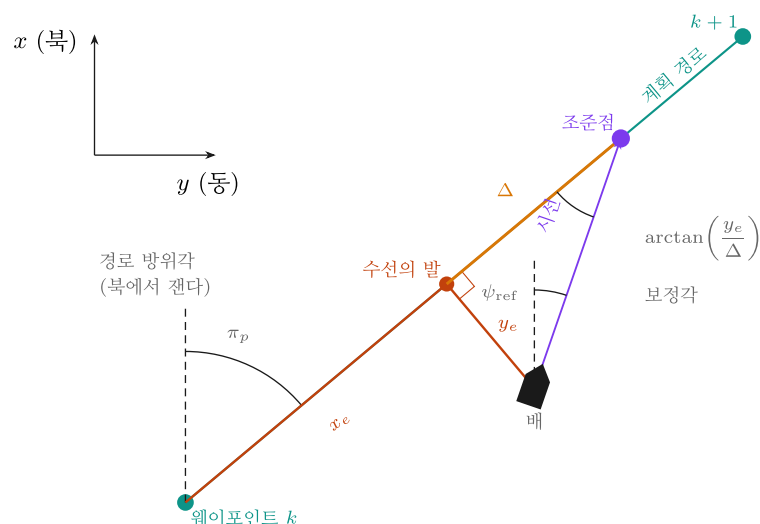
식이 맞는지는 특수한 위치 세 곳에 넣어 보면 바로 알

| 배의 위치                   | $x_e$       | $y_e$          |
|-------------------------|-------------|----------------|
| 웨이포인트 $k$ 위             | 0           | 0              |
| 웨이포인트 $k + 1$ 위         | $d$ (구간 길이) | 0              |
| 경로에서 우현으로 $\varepsilon$ | 그대로         | $+\varepsilon$ |

셋 다 맞지 않으면  $\pi_p$  의 인자 순서나 부호가 틀린 것임

## 1-6. LOS 유도법칙

- 요지: 웨이포인트가 아니라 **경로 위의 한 점을** 겨냥함
  - 그 점은 경로에서 배와 가장 가까운 점보다  $\Delta$  만큼 앞선 점
- 이름의 뜻: LOS = Line Of Sight, **시선(視線)**. 눈으로 목표를 쫓는 것과 같음



## 그림 읽는 법

| 기호                  | 뜻                                              |
|---------------------|------------------------------------------------|
| $\pi_p$             | 경로 방위각. 북에서 잼                                  |
| $x_e$               | 경로 방향 오차 — 구간을 얼마나 왔는가                         |
| 수선의 발               | 경로 위에서 배와 가장 가까운 점. 배에서 경로에 수직으로 내린 발          |
| $y_e$               | 경로 이탈량. 수선의 발에서 배까지의 부호 있는 거리 (우현 +)           |
| $\Delta$            | lookahead distance — 수선의 발에서 경로를 따라 얼마나 앞을 보는가 |
| 조준점                 | 수선의 발보다 $\Delta$ 앞선 점. 언제나 경로 위에 있음            |
| 시선                  | 배에서 조준점으로 향하는 직선. 이 방향이 곧 지령임                  |
| $\psi_{\text{ref}}$ | 그 시선의 방위각. heading 제어기로 보내는 값                  |

## 식이 나오는 자리

- 경로 좌표계에서 보면 세 점의 자리가 아주 단순함

| 점     | 경로 좌표계에서의 위치        |
|-------|---------------------|
| 배     | $(x_e, y_e)$        |
| 수선의 발 | $(x_e, 0)$          |
| 조준점   | $(x_e + \Delta, 0)$ |

- 배에서 조준점으로 가는 벡터는 빼기 한 번임

$$\text{조준점} - \text{배} = \begin{bmatrix} (x_e + \Delta) - x_e \\ 0 - y_e \end{bmatrix} = \begin{bmatrix} \Delta \\ -y_e \end{bmatrix}$$

- 이 벡터가 경로에서 얼마나 틀어져 있는가가 보정각임

$$\text{atan2}(-y_e, \Delta) = -\arctan\left(\frac{y_e}{\Delta}\right) \quad (\Delta > 0 \text{ 이므로})$$

- 경로 자체가 북에서  $\pi_p$  만큼 돌아가 있으므로, 둘을 더하면 북 기준 지령이 됨

$$\boxed{\psi_{\text{ref}} = \pi_p - \arctan\left(\frac{y_e}{\Delta}\right)}$$

- 두 항의 역할
  - $\pi_p$ : 경로와 나란히 서라
  - $\arctan$  항: 벗어난 만큼 경로 쪽으로 기울여라

## 그림의 숫자로 확인

- 그림은  $\pi_p = 50^\circ$ ,  $y_e = 6 \text{ m}$ ,  $\Delta = 10 \text{ m}$  로 그렸음

$$\arctan\left(\frac{6}{10}\right) = 30.96^\circ \quad \Rightarrow \quad \psi_{\text{ref}} = 50^\circ - 30.96^\circ = 19.04^\circ$$

- 배가 경로의 오른쪽에 있으므로 지령이  $\pi_p$  보다 왼쪽으로 나옴. 부호가 맞음

## 극한 네 가지 — 이 법칙이 안전한 이유

| 조건                          | 결과                                                                                         | 뜻                                    |
|-----------------------------|--------------------------------------------------------------------------------------------|--------------------------------------|
| $y_e \rightarrow 0$         | $\psi_{\text{ref}} \rightarrow \pi_p$                                                      | 경로 위에서는 경로와 나란히. 보정이 <b>부드럽게</b> 사라짐 |
| $y_e \rightarrow +\infty$   | $\psi_{\text{ref}} \rightarrow \pi_p - 90^\circ$                                           | 우현으로 아무리 멀어도 <b>경로를 정면으로</b> 향함      |
| $\Delta \rightarrow 0$      | $\psi_{\text{ref}} \rightarrow \pi_p \mp 90^\circ$ ( $y_e > 0$ 이면 $-$ , $y_e < 0$ 이면 $+$ ) | 아무리 작은 이탈에도 최대로 꺾음 — 커짐/꺼짐 제어가 됨     |
| $\Delta \rightarrow \infty$ | $\psi_{\text{ref}} \rightarrow \pi_p$                                                      | 보정을 아예 하지 않음 — heading 유지가 됨         |

### ★ arctan 이라서 안전함

$|\arctan(\cdot)| < 90^\circ$  이므로 지령은 **경로 방향에서  $90^\circ$  를 절대 넘지 않음**

아무리 멀리 벗어난 곳에서 출발해도 경로에서 **멀어지라는 지령은 나오지 않음**

$\psi_{\text{ref}} = \pi_p - K y_e$  처럼 비례로만 만들면 이 보장이 없음.  $y_e$  가 크면

한 바퀴를 넘어가는 지령이 나옴

### ! $\Delta$ 는 배가 아니라 수선의 발에서 잼

조준점은 배에서  $\Delta$  떨어진 점이 아니라, 수선의 발에서 경로를 따라  $\Delta$  앞선 점임

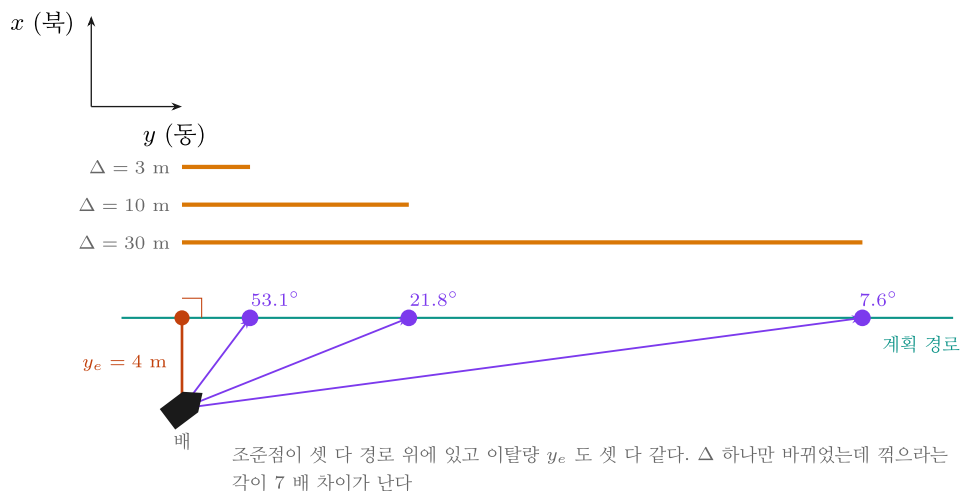
배에서 조준점까지의 실제 거리는  $\sqrt{\Delta^2 + y_e^2}$  이고  $\Delta$  보다 큼

배에서  $\Delta$  를 재면 보정각이  $\arcsin(y_e/\Delta)$  가 됨.  $y_e \rightarrow 0$ ,  $\Delta \rightarrow \infty$  극한은 그대로지만,

$|y_e| > \Delta$  이면 조준점이 존재하지 않아 **법칙 자체가 정의되지 않음**

## 1-7. lookahead distance $\Delta$

- LOS 법칙에서 **자유롭게 고를 수 있는 값은  $\Delta$  하나뿐임**. 나머지는 경로와 위치가 정함
- $\Delta$  는 분모임. 같은 이탈량이어도  $\Delta$  가 작으면 크게 꺾음

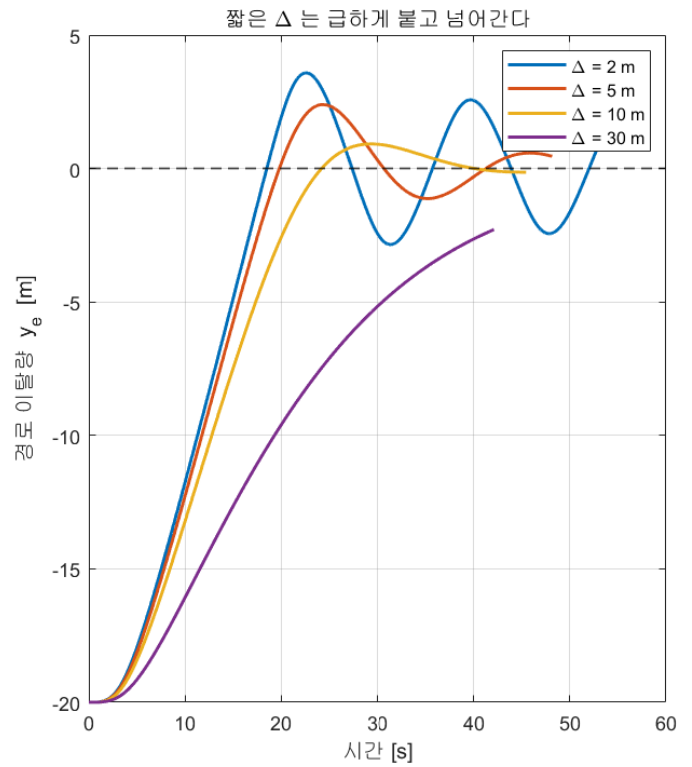
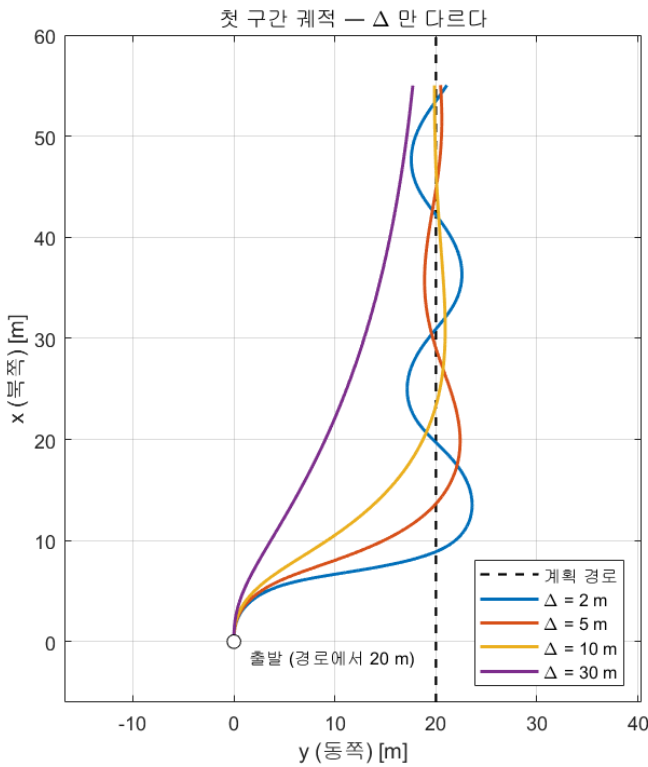


- 그림의 세 경우는 **이탈량이 똑같음**.  $\Delta$  만 바뀌었는데 꺾으라는 각이  **$53.1^\circ$  에서  $7.6^\circ$  로 일곱 배 차이가 남**

## 눈으로 보기 — 같은 자리에서 출발해 $\Delta$ 만 바꾼다

```
>> W07_setup
>> W07_lookahead_demo
```

- 네 번 모두 **같은 자리에서 같은 만큼(20 m) 벗어난 채** 출발함.  $\Delta$  말고는 아무것도 바꾸지 않음
- 첫 구간(북쪽 80 m)만 잘라서 그림. 구간이 바뀌면 전환과 선회가 섞여  $\Delta$  의 성격이 묻힘



#### 측정값 — 첫 구간에서의 경로 복귀 (조류 없음)

| $\Delta$ [m] | $ y_e  < 1$ m 진입 [s] | 오버슈트 [m] | 경로 가로지름 [회, 첫 구간] |
|--------------|----------------------|----------|-------------------|
| 2            | 17.8                 | 3.584    | 5                 |
| 5            | 18.9                 | 2.395    | 3                 |
| 10           | 22.2                 | 0.921    | 2                 |
| 30           | 진입 못 함               | 0.000    | 0                 |

- $\Delta$  가 짧을수록 급하게 꺾어 들어옴.  $\Delta = 2$  m 는 가장 빨리(17.8 s) 경로에 닿음
- 그 대가가 오른쪽 그림에 그대로 보임.  $\Delta = 2$  m 는 경로를 넘어 반대쪽으로 3.58 m 까지 튀었다가 돌아오고, 첫 구간 안에서만 경로를 다섯 번 가로지름
- $\Delta = 30$  m 는 정반대임. 한 번도 넘지 않지만(오버슈트 0) 첫 구간이 끝날 때까지 경로에 닿지도 못함
- $\Delta = 10$  m 는 한 번만 살짝 넘고(0.92 m) 곧 붙음

#### ★ 이 그림이 $\Delta$ 의 전부임

짧은  $\Delta$  = 빨리 붙는 대신 넘어감. 긴  $\Delta$  = 넘어가지 않는 대신 못 붙음

둘 다 좋게 만드는 값은 없음. 그래서  $\Delta$  는 정답을 찾는 게 아니라 고르는 값임

조류를 켜면 이 균형점이 짧은 쪽으로 옮겨 감. 아래 두 번째 표(미션 전체 기준)가 그것을 숫자로 보여 줌

- `W07_lookahead_demo` 는 안에서 `W07_setup` 을 다시 부르며 워크스페이스를 지움
- 명령 창에서 `current_speed = 0.4` 만 치면 무시됨. `W07_setup.m` 파일에서 `current_speed = 0.4` , `current_direction = 90` 으로 고친 뒤 다시 돌릴 것

#### 실제로 재 보면

- 본 과목의 오프라인 모델( `W07_0_offline` )로  $\Delta$  만 바꿔 가며 미션 전체를 돌린 값임
- 웨이포인트 4점 사각형(한 변 80 m),  $u_{ref} = 1.5$  m/s,  $R = 5$  m, 출발점은 경로에서 20 m 벗어난 곳
- 평균 이탈량은 `W07_plot` 이 찍는 초기 60초 제외 값 ( `S.mean_ye_ss` ). 전 구간 평균이 아님
- 아래 두 표는 `W07_sweep('A')` 로 그대로 다시 나옴 (2026-09-19 재실행, 모든 칸 같음)

## 조류 없음

| $\Delta$ [m] | 수렴 [s] | 정착 후 평균 $ y_e $ [m] (초기 60 s 제외) | 평균 $ r $ [°/s] | 경로 가로지름 [회] | 완주 [s] |
|--------------|--------|----------------------------------|----------------|-------------|--------|
| 2            | 17.8   | 1.706                            | 10.705         | 22          | 226.1  |
| 3            | 18.1   | 1.323                            | 7.001          | 17          | 219.8  |
| 5            | 18.9   | 0.866                            | 3.393          | 16          | 211.2  |
| <b>10</b>    | 22.2   | <b>0.700</b>                     | 1.947          | 11          | 206.5  |
| 20           | 35.6   | 0.949                            | 1.567          | 4           | 203.2  |
| 30           | 미수렴    | 1.328                            | 1.450          | 4           | 203.1  |

- 수렴 시각은  $\Delta$  와 함께 계속 늘어남.  $\Delta = 30$  m 는 첫 구간이 끝날 때까지 1 m 안에 들어오지 못함
- 평균 이탈량만 가운데가 가장 작음.  $\Delta = 10$  m 가 최저점이고, 양쪽으로 가면 나빠짐
- 평균 요각속도와 경로 가로지름 횟수는  $\Delta$  와 함께 계속 줄어듦.  $\Delta = 2$  m 는  $\Delta = 30$  m 보다 평균 요각속도가 7.4 배 크고(그만큼 요 모멘트를 더 씀) 경로를 5.5 배 더 많이 가로지름 — 이것이 "지그재그" 의 정체임
- 완주 시각도 계속 줄어듦. 흔들리며 가면 그만큼 늦게 도착함

## 조류 0.4 m/s (동쪽으로 흐름, 90°)

| $\Delta$ [m] | 수렴 [s] | 정착 후 평균 $ y_e $ [m] (초기 60 s 제외) | 평균 $ r $ [°/s] | 경로 가로지름 [회] | 완주 [s] |
|--------------|--------|----------------------------------|----------------|-------------|--------|
| 2            | 16.2   | 1.902                            | 10.814         | 21          | 229.7  |
| 3            | 16.3   | 1.438                            | 6.862          | 18          | 219.8  |
| <b>5</b>     | 16.7   | <b>1.182</b>                     | 3.386          | 13          | 211.7  |
| 10           | 18.2   | 1.504                            | 1.882          | 9           | 207.0  |
| 20           | 21.7   | 2.486                            | 1.467          | 6           | 204.0  |
| 30           | 25.0   | 3.245                            | 1.351          | 4           | 204.4  |

### ★ 조류가 있으면 최적 $\Delta$ 가 작아짐

조류가 없을 때는  $\Delta = 10$  m 가 최저점이었는데, 조류가 생기자  $\Delta = 5$  m 로 옮겨 갔음  
그리고 큰  $\Delta$  일수록 오차가 훨씬 크게 늘어남 —  $\Delta = 30$  m 는 3.245 m 로 최저점의 약 2.7 배임  
이유는 바로 아래에서 식 하나로 나옴

## 조류 속에서는 경로 옆에 붙어 간다 — $y_e = \Delta \tan \beta$

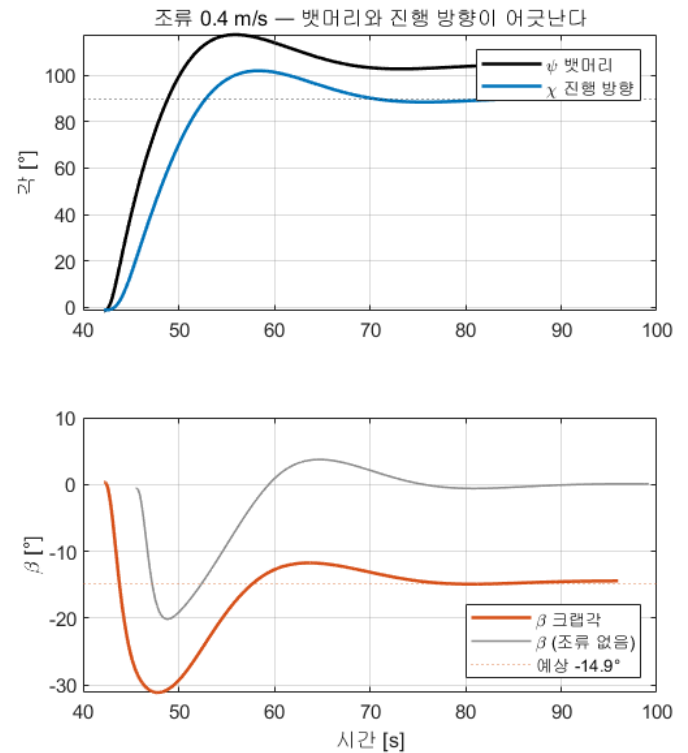
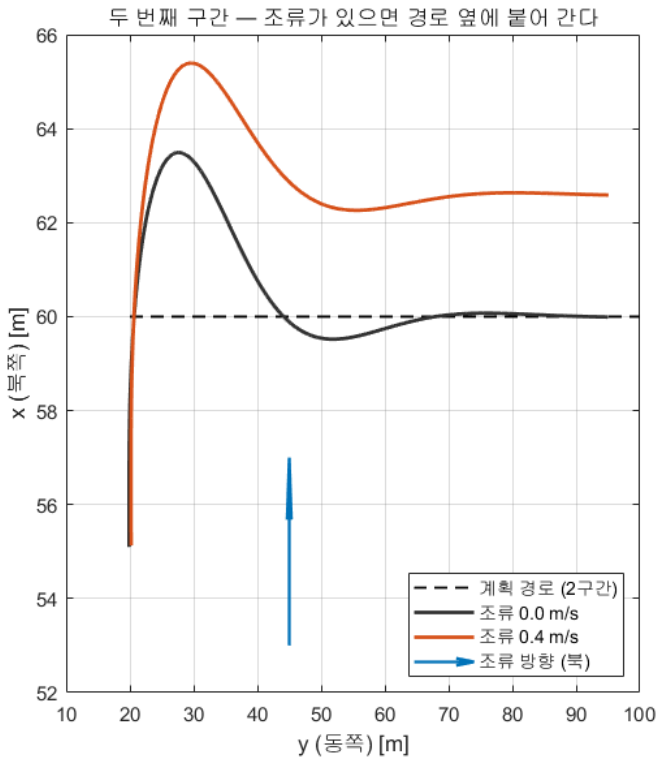
- 조류가 경로를 가로질러 밀면, 배는 뱃머리를 조류 쪽으로 틀어야 경로와 나란히 갈 수 있음. 그 각이 크랩각  $\beta$  임 (1-2절)
- 그런데 LOS 는 선수각 지령을 냄. 정상상태에서 진행 방향이 경로 방향이 되려면  $\psi = \pi_p - \beta$  여야 하고, LOS 가 그 값을 내려면

$$\pi_p - \arctan\left(\frac{y_e}{\Delta}\right) = \pi_p - \beta \quad \Rightarrow \quad \boxed{y_e = \Delta \tan \beta}$$

- 결과: 경로에서  $\Delta \tan \beta$  만큼 떨어진 채 나란히 감
  - $\beta$  는 조류가 정함
  - $\Delta$  가 클수록 그만큼 멀리 떨어짐 — 위 표에서 큰  $\Delta$  의 오차가 크게 늘어나는 이유
- 대학원 강의 W04 §4-7 과 같은 식임

>> W07\_crab\_demo

- 두 번째 구간(정동)에 북쪽으로 흐르는 조류 0.4 m/s 를 줌. 구간을 정면으로 가로지름



- 정상 출력 (구간 뒤쪽 절반의 평균)

| 조류[m/s] | psi[°] | chi[°] | beta[°] | beta 예상[°] | y_e[m] | Delta*tan(b)[m] |
|---------|--------|--------|---------|------------|--------|-----------------|
| 0.0     | 89.51  | 89.38  | -0.12   | -0.00      | 0.043  | -0.021          |
| 0.4     | 104.02 | 89.58  | -14.44  | -14.93     | -2.518 | -2.575          |

| 읽는 법                                   | 뜻                                                                 |
|----------------------------------------|-------------------------------------------------------------------|
| $\psi = 104^\circ, \chi = 89.6^\circ$  | 뱃머리는 남쪽으로 $14^\circ$ 틀었는데, 배는 거의 정동( $90^\circ$ )으로 갑             |
| $\beta = -14.44^\circ$                 | $\text{atan}(0.4/1.5) = 14.93^\circ$ 에 가까움. 옆으로 밀리는 속도가 전부 조류에서 옴 |
| $y_e = -2.518 \text{ m}$               | 경로에서 북쪽(좌현)으로 2.5 m 떨어져 나란히 갑. 궤적 그림의 주황 선                        |
| $\Delta \tan \beta = -2.575 \text{ m}$ | 식이 예측한 값. 측정과 약 2 % 차이로 맞음                                        |

### 이 오차를 없애려면

원인이 "선수각을 지령했다" 는 데 있으므로, 크랩각만큼 미리 틀어 주면 됨 —  $\psi_{\text{ref}} = \chi_d - \beta$   
 8주차 1-8절이 로이더링에서 이것을 해서 반경 오차를 42 % 줄임.  $\beta$  를 모를 때 적분으로 지우는 방법(ILOS)은 대학원 강의 W04 §4-8 에 있음

## 정리

| $\Delta$    | 빨리 붙는가 | 요 모멘트를 적게 쓰는가 | 조류에 강한가                                                                                         |
|-------------|--------|---------------|-------------------------------------------------------------------------------------------------|
| 작음 (2~3 m)  | O      | X — 지그재그      | $\Delta$ — 정상 오프셋 $\Delta \tan \beta$ 는 작지만 지그재그로 평균 이탈은 큼 ( $\Delta = 2 \text{ m}$ 에서 1.902 m) |
| 적당 (5~10 m) | 보통     | 보통            | 보통                                                                                              |
| 큼 (20~30 m) | X      | O             | X — 옆 오차가 크게 남음                                                                                 |

- 어느 값도 세 가지를 다 잘하지 못함. 그래서 이것은 정답이 있는 튜닝이 아니라 맞바꿈(trade-off)임
- 흔한 기준은 배 길이의 2~5 배임. WAM-V 는 약 4.9 m 이므로 10~25 m
- 본 과목의 기본값은  $\Delta = 10 \text{ m}$  임. 위 표의 첫 번째 조건에서 최저점이고, 기준 범위의 아래쪽 끝임

### 더 나아가기 — $\Delta$ 를 고정하지 않는 방법

멀리 있을 때는 작게(공격적으로), 가까이 붙으면 크게(부드럽게) 만드는 방법이 있음

$$\Delta(y_e) = (\Delta_{\max} - \Delta_{\min})e^{-y_e^2} + \Delta_{\min}$$

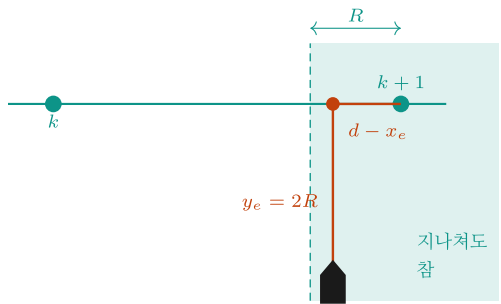
참고 자료( 2. USV 제어기 설계 , 2025)의 측정값으로는 고정  $\Delta$  가 1.6763 / 1.9824 / 2.3202 m ( $\Delta = 1.5 / 6 / 8 \text{ m}$ ) 인 조건에서 가변  $\Delta$  가 1.5875 m 로 셋 모두보다 좋았음  
본 과목 모델에는 넣지 않았음. 과제 ②의 확장 주제로 쓸 수 있음

## 1-8. 웨이포인트 전환 — 언제 다음 구간으로 넘기는가

- 구간은 끝나야 함. 무엇을 보고 끝났다고 판단할지가 남았고, 널리 쓰이는 방법이 두 가지인데 둘은 같지 않음

경로 방향 판정

남은 경로 방향 거리가  $R$  보다 작은가



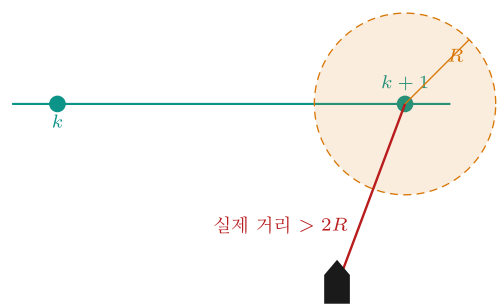
$d - x_e < R$  이므로

전환한다

경로 방향 판정:  $d - x_e < R$

수락 원 판정

실제 직선거리가  $R$  보다 작은가



원 밖이므로

전환하지 않는다 — 계속 기다린다

수락 원 판정:  $\|\mathbf{p} - \mathbf{p}_{k+1}\| \leq R$

| 기호        | 뜻               | 값                      |
|-----------|-----------------|------------------------|
| $d$       | 지금 구간의 길이       | m — 본 과목은 네 구간 모두 80 m |
| $x_e$     | 경로 방향 오차 (§1-5) | m                      |
| $d - x_e$ | 구간에 남은 경로 방향 거리 | m                      |
| $R$       | 수락반경            | m — $R_{LOS}$ , 기본값 5  |

### 둘은 언제 갈라지는가

- 경로 위에 있으면 둘은 같음.  $y_e = 0$  이면 남은 거리가 곧 직선거리이기 때문임
- 벗어나 있으면 다름. 피타고라스로 바로 나눔

$$\|\mathbf{p} - \mathbf{p}_{k+1}\| = \sqrt{(d - x_e)^2 + y_e^2} \geq d - x_e$$

- 실제 거리는 남은 거리보다 작아질 수 없음
  - 따라서 수락 원 판정이 언제나 더 엄격함.  $y_e$  가 클수록 차이가 커짐
- 그림의 배는 남은 거리가  $R$  보다 작는데 이탈량이  $2R$  임. 경로 방향 판정은 넘어가고, 수락 원 판정은 넘어가지 않음

## 실제로 재 보면 — 수락 원은 걸릴 수 있다

- 같은 모델, 같은 미션에서 두 판정을 바꿔 가며 돌린 값임. 조류는 **동쪽으로 흐름** (90°)
- 재현: `w07_sweep('B')` (이 표), `w07_sweep('C')` (아래 여덟 방향)

| $R$ [m]  | 판정          | 조류 [m/s]   | 정착 후 평균 $ y_e $ [m] (초기 60 s 제외) | 완주 [s]     | 전환 횟수    |
|----------|-------------|------------|----------------------------------|------------|----------|
| 5        | 경로 방향       | 0.0        | 0.700                            | 206.5      | 3        |
| 5        | 수락 원        | 0.0        | 0.700                            | 206.5      | 3        |
| 5        | 경로 방향       | 0.4        | 1.504                            | 207.0      | 3        |
| 5        | 수락 원        | 0.4        | 1.526                            | 207.7      | 3        |
| 2        | 경로 방향       | 0.4        | 1.698                            | 214.1      | 3        |
| <b>2</b> | <b>수락 원</b> | <b>0.4</b> | 2.582                            | <b>미완주</b> | <b>0</b> |
| 2        | 경로 방향       | 0.8        | 2.698                            | 211.9      | 3        |
| <b>2</b> | <b>수락 원</b> | <b>0.8</b> | 4.807                            | <b>미완주</b> | <b>0</b> |

- 조류가 없으면 두 판정이 **완전히 같은 결과**를 냄. 예상대로임
- $R$  이 넉넉하면(5 m) 조류가 있어도 둘 다 통과함
- $R$  을 2 m 로 줄이고 조류를 주면 수락 원 판정은 첫 구간 끝의 WP2 (60, 20) 에서 전환하지 못하고 미션이 끝나지 않음
- 조류 방향을 45° 씩 여덟 방향으로 돌려 봐도( $R = 2$  m, 0.8 m/s) 여덟 번 모두 수락 원은 미완주, 경로 방향은 모두 완주했음 (202.0 ~ 211.9 s)

### ⚠ 수락 원이 왜 걸리는가

조류에 밀린 배는 웨이포인트 옆을 스쳐 지나감. 옆으로  $y_e$  만큼 벗어난 채 지나가면 그 점까지의 최소 거리가  $y_e$  보다 작아질 수 없음.  $y_e > R$  이면 원 안에 들어가는 순간이 영영 오지 않음

그 뒤 배가 어떻게 되는지는 유도법칙이 정함

- LOS 지령  $\pi_p - \arctan(y_e/\Delta)$  에는  $x_e$  가 없음 → 웨이포인트를 지나쳐도 되돌아가지 않음
- 배는 구간의 연장선을 따라 계속 북쪽으로 감 (VRX 라면  $x = 60$  m 너머는 육지)
- 표의 미완주 행 평균 이탈량 2.582 m 가 정상 오프셋  $\Delta \tan \beta$  수준인 것도 선을 계속 따라가고 있다는 증거
- 지나친 웨이포인트로 되돌아가려는 것은 웨이포인트를 겨냥하는 atan2 유도일 때뿐

경로 방향 판정에는 이 문제가 없음. 옆으로 얼마나 벗어났든 구간을 다 달렸으면

넘어가기 때문임. MSS 툴박스의 원본 `GNC/LOSchi.m` · `ILOSpsi.m` 도 이 판정( $d - x_e < R_{\text{switch}}$ )을 씀  
배포 폴더 `w07_matlab/LOSchi.m` 은 연구실 수정본이라 이 부분이 수락 원 판정으로 바뀌어 있음 (원본 판정은 주석으로 남아 있음)

본 과목의 기본값도 `sw_mode = 1` (경로 방향) 임. 실습 D-3 에서 직접 확인함

## $R$ 은 얼마로 두는가

- $R$  은 튜닝 값이지만 넘으면 안 되는 한계가 하나 있음

$$R < \min_k d_k$$

- $R$  이 가장 짧은 구간보다 클 때
  - 그 구간이 시작되는 순간에 이미 전환 조건이 만족됨
  - 결과: 한 걸음도 달리지 않고 웨이포인트를 건너뛸

- 본 과목은 가장 짧은 구간이 80 m 이고  $R = 5$  m 이므로 여유가 큼

$R$  을 바꿔 가며 잔 값 ( $\Delta = 10$  m, 조류 없음)

| $R$ [m] | 모서리 컷 [m] | 완주 [s] |
|---------|-----------|--------|
| 2       | 0.08      | 213.7  |
| 5       | 0.70      | 206.5  |
| 10      | 3.09      | 196.0  |
| 20      | 7.65      | 179.8  |

- "모서리 컷" 은 배가 모서리 웨이포인트(WP2 · WP3 · WP4)에 가장 가까이 간 거리이고, 표는 세 모서리 중 **가장 크게 자른 값**임. 재현: `w07_sweep('D')`
  - $R = 5$  m 의 0.70 m 가 정착 후 평균  $|y_e|$  0.700 m 와 같은 것은 우연 (세 모서리 0.59 · 0.70 · 0.69 m)
- $R$  이 크면 **미리 돌아** 모서리를 잘라먹고, 그만큼 일찍 도착함
- $R$  이 작으면 구간 끝까지 달렸다가 급하게 뛴. 모서리는 정확하지만 회전이 급해짐
- 어느 쪽이 맞는지는 **미션이 정함**. 모서리 정확도가 중요한가, 급회전을 피하는 것이 중요한가

### 미션 종료

- 마지막 구간에는 넘어갈 다음 웨이포인트가 없음
  - 아무 처리도 하지 않으면 배는 **마지막 구간의 연장선을 따라 계속 감**
- 본 과목의 처리: `gate` 출력
  - 마지막 웨이포인트에서 전환 조건이 만족되면 `wp_idx` 를  $n$  ("완주")으로 올리고 `gate = 0` → 속도 지령 0
  - `wp_idx` 는 한 스텝 늦게 되돌아와 다음 계산의 `idx_prev` 가 되므로 **한 번  $n$  이 되면 계속  $n$  — 조류에 밀려 결승선 뒤로 돌아가도 다시 출발하지 않음 (래치)**
- **유도법칙이 아니라 미션의 룰임**
  - 제자리 유지 · 주위 선회 · 추진기 정지 중 무엇을 할지는 임무가 정함
  - 9주차에서 Stateflow 가 이 판단을 맡음

## 1-9. 수식과 코드 — 한 줄씩 맞춰 보기

- 지금까지의 식이 모델 안에서 어떤 코드가 되는지 **한 줄씩** 대응시킨 표임
- 코드는 `Guidance` 서브시스템 안의 MATLAB Function 블록 `GuidanceLaw` 에 그대로 들어 있음
- 실습 B 에서 블록을 열어 이 표와 나란히 놓고 볼 것

| 하는 일                | 수식                                                                   | 코드                                                                                                                                        |
|---------------------|----------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------|
| 지금 구간<br>의 양 끝      | $\mathbf{p}_k, \mathbf{p}_{k+1}$                                     | <code>xk = wp_x(k); yk = wp_y(k);<br/>xk1 = wp_x(k+1); yk1 = wp_y(k+1);</code>                                                            |
| 경로 방위<br>각          | $\pi_p = \text{atan2}(y_{k+1} - y_k, x_{k+1} - x_k)$                 | <code>pi_p = atan2(yk1 - yk, xk1 - xk);</code>                                                                                            |
| 경로 방향<br>오차         | $x_e = (x - x_k) \cos \pi_p + (y - y_k) \sin \pi_p$                  | <code>x_e = (x_n - xk)*cos(pi_p) + (y_n -<br/>yk)*sin(pi_p);</code>                                                                       |
| 경로 이탈<br>량          | $y_e = -(x - x_k) \sin \pi_p + (y - y_k) \cos \pi_p$                 | <code>y_e = -(x_n - xk)*sin(pi_p) + (y_n -<br/>yk)*cos(pi_p);</code>                                                                      |
| atan2 유<br>도 (§1-3) | $\psi_{\text{ref}} = \text{atan2}(y_{k+1} - y, x_{k+1} - x)$         | <code>psi_ref = atan2(yk1 - y_n, xk1 - x_n);</code>                                                                                       |
| LOS 유도<br>(§1-6)    | $\psi_{\text{ref}} = \pi_p - \arctan\left(\frac{y_e}{\Delta}\right)$ | <code>psi_ref = pi_p - atan(y_e / Delta);</code>                                                                                          |
| 구간 길이               | $d = \ \mathbf{p}_{k+1} - \mathbf{p}_k\ $                            | <code>d = sqrt((xk1 - xk)^2 + (yk1 - yk)^2);</code>                                                                                       |
| 전환 —<br>경로 방향       | $d - x_e < R$                                                        | <code>reached = (d - x_e) &lt; R;</code>                                                                                                  |
| 전환 —<br>수락 원        | $\ \mathbf{p} - \mathbf{p}_{k+1}\  \leq R$                           | <code>reached = sqrt((xk1 - x_n)^2 + (yk1 - y_n)^2)<br/>&lt;= R;</code>                                                                   |
| 인덱스 올<br>리기         | $k \leftarrow k + 1$                                                 | <code>elseif reached &amp;&amp; (k &lt; n-1), wp_idx = k + 1;</code>                                                                      |
| 미션 종료               | 마지막 구간에서 도달 $\Rightarrow$ 정지, 한 번 끝나면<br>계속 정지                       | <code>elseif reached &amp;&amp; (k == n-1), wp_idx = n; · if<br/>idx_prev &gt;= n, wp_idx = n; · gate =<br/>double(wp_idx &lt; n);</code> |

- 기호와 코드 이름의 대응:  $x \leftrightarrow x_n$ ,  $y \leftrightarrow y_n$  (배의 북쪽 · 동쪽 위치),  $x_k \leftrightarrow xk$ ,  $y_k \leftrightarrow yk$ ,  $x_{k+1} \leftrightarrow xk1$ ,  $y_{k+1} \leftrightarrow yk1$
- 코드의  $n$  = 웨이포인트 개수. 구간은  $n-1$  개이므로 마지막 구간의 번호가  $k = n-1$  임

#### ★ 코드에는 위첨자를 쓸 수 없음

교재는 오차가 **경로 좌표계**의 값임을  $x_e^p, y_e^p$  처럼 위첨자로 밝힘

MATLAB 식별자에는 위첨자를 붙일 자리가 없어서 코드는  $x_e, y_e$  로만 씀

**모델 안의  $y_e$  는 언제나 경로 좌표계의 값이지 동쪽 방향 오차가 아님**

이 둘을 섞는 것이 이 주차에서 가장 혼한 착각이고, 두 값은  $\pi_p$  만큼 다름

### ! MSS 툴박스와 변수 이름이 조금 다름

MSS 원본 `GNC/LOSchi.m` · `ILOSpsi.m` · `crosstrackwpt.m` 과 맞춰 읽을 때 아래 표로 바꿔 읽음

| 뜻                  | MSS 원본                              | 본 과목 ( <code>W07_setup</code> · <code>GuidanceLaw</code> ) |
|--------------------|-------------------------------------|------------------------------------------------------------|
| 경로 방위각             | <code>pi_h</code>                   | <code>pi_p</code>                                          |
| 경로 방향 오차 · 경로 이탈량  | <code>x_e</code> · <code>y_e</code> | <code>x_e</code> · <code>y_e</code> (같은)                   |
| lookahead distance | <code>Delta_h</code>                | <code>Delta</code>                                         |
| 전환 거리 (수락반경)       | <code>R_switch</code>               | <code>R_LOS</code> (설정 파일), <code>R</code> (블록 안)          |

- 식의 형태(회전 두 줄,  $\pi - \arctan(y_e/\Delta)$ )는 MSS 와 같음
- 배포 폴더 `W07_matlab/LOSchi.m` 은 연구실 수정본임 — `pi_p` · `Delta` · `R_switch` 를 쓰고, 전환 판정 이 수락 원임 (§1-8)

## 2부 · 실습

### ★ 두 모델의 관계를 먼저 이해할 것

| 모델                | 운동모델                 | 조류 | Gazebo |
|-------------------|----------------------|----|--------|
| W07_0_offline.slx | Simulink 안의 3자유도 방정식 | 가능 | 불필요    |
| W07_1_vrx.slx     | Gazebo 의 WAM-V       | 불가 | 필요     |

유도부·내부루프·추진기·제인은 두 모델이 완전히 같음. 운동모델만 바뀜

### 실행하면 이런 결과가 나온다

#### ★ 아래는 기준 환경에서 실제로 받은 출력임

값이 크게 다르면 설정 파일을 먼저 확인함. 소수점 마지막 자리는 달라질 수 있음

```
cd('<배포 폴더>/W07_simulink')
W07_setup
```

- W07\_setup 이 설정 후 오프라인 모델을 바로 실행하고 그림까지 그림 ( auto\_run = 1 ). sim 을 따로 부를 필요 없음
- 정상 출력

W07 설정 완료

웨이포인트 5개, 유도 = LOS

Delta = 10 m, R = 5 m, u\_ref = 1.5 m/s

전환 판정 = 경로 방향 (d - x\_e < R)

조류 = 0 m/s @ 0 deg (오프라인 모델에서만 적용)

오프라인 모델 실행 중 (auto\_run = 0 으로 두면 실행하지 않는다)

실시간 그림 켜짐 - 새 창이 열린다. 끄려면 animate = 0

===== LOS 유도 / 조류 0 m/s =====

평균 |y\_e| 1.975 m (전 구간)

평균 |y\_e| 0.700 m (초기 60초 제외) <- 과제에 쓸 값

최대 |y\_e| 20.000 m

완주 시각 206.5 s

평균 |크랩각| 2.97 deg

정상상태 속도 1.500 m/s (목표 1.50)

| 읽는 법                          | 뜻                                                |
|-------------------------------|--------------------------------------------------|
| 전 구간 1.975 m vs 초기 제외 0.700 m | 출발 직후 경로에 붙는 동안의 큰 오차가 평균을 끌어올림. 과제에는 초기 제외 값을 씀 |
| 최대 20.000 m                   | 첫 구간 시작점이 경로에서 20 m 떨어져 있음. 오차가 아니라 초기 조건임       |
| 평균 크랩각 2.97°                  | 조류가 0인데도 0이 아님. 선회 중 생기는 sway 때문임                |
| 정상상태 속도 1.500                 | 목표와 일치. 속도 루프가 정상이라는 뜻                           |

## ⚠ 창이 하나 더 열리면서 배가 움직이는 그림이 나옴

`animate = 1` 이라서 그러함. 느리면 `w07_setup` 안에서 `animate = 0` 으로 끄

## A. 설정 파일 열어 보기 (10분)

```
cd('<배포 폴더>/W07_simulink')
edit W07_setup.m
```

- 이 파일 하나만 고치면 됨. Simulink 의 Constant 블록들이 여기서 만든 변수 이름을 그대로 참조함
- 연구실의 기존 모델(`VRX_tilt4_controller_full.slx`)도 똑같은 방식임

### 학생이 바꾸는 값

```
wp_north = [ -20  60  60 -20 -20 ]; % 웨이포인트 x (북쪽) [m]
wp_east  = [  20  20 100 100  20 ]; % 웨이포인트 y (동쪽) [m]

guidance_mode = 2; % 1 = atan2, 2 = LOS
Delta         = 10; % lookahead distance [m]
R_LOS         = 5; % 수락반경 [m]
sw_mode       = 1; % 전환 판정. 1 = 경로 방향, 2 = 수락 원
u_ref         = 1.5; % 목표 속도 [m/s]

current_speed  = 0.0; % 조류 [m/s] (오프라인 모델에서만)
current_direction = 0.0; % 조류 방향 [deg]

Kp_psi = 400; Kd_psi = 200; % heading P-D (wn 0.78, zeta 0.98 - 6주차 1-7절)
Kp_u   = 300; Ki_u   = 100; % 속도 PI
F_max  = 500; % 추력 포화 [N]
d_mode = 1; % 1 = 요각속도 되먹임, 2 = 오차 미분
```

### i 왜 배가 경로 위에서 출발하지 않는가

배는  $(0, 0)$  에서 출발하는데 첫 구간은  $y = 20$  m (동쪽 20 m) 선임. 20 m 벗어난 채로 시작함  
일부러 그렇게 두었음. 경로 위에서 출발하면 두 유도법칙의 차이가 거의 보이지 않음

### ✗ 좌표 범위를 크게 벗어나면 배가 육지에 얹힘

`sydney_regatta` 에서 스폰 기준으로 확인된 수역은 대략  
 $x \in [-28, 60], y \in [-18, 100]$  임  
(VRX `wayfinding_task` 의 공식 웨이포인트 3점을 환산해 얻은 값)

### 실행 — 이것만 하면 된다

```
W07_setup
```

### ★ 설정 파일을 실행하면 곧바로 시뮬레이션이 됨

모델을 따로 열고 Run 을 누를 필요가 없음  
설정만 바꾸고 직접 Run 하고 싶으면 `auto_run = 0` 으로 뒀

• 정상 출력

W07 설정 완료

웨이포인트 5개, 유도 = LOS  
Delta = 10 m, R = 5 m, u\_ref = 1.5 m/s  
전환 판정 = 경로 방향 (d - x\_e < R)  
조류 = 0 m/s @ 0 deg (오프라인 모델에서만 적용)

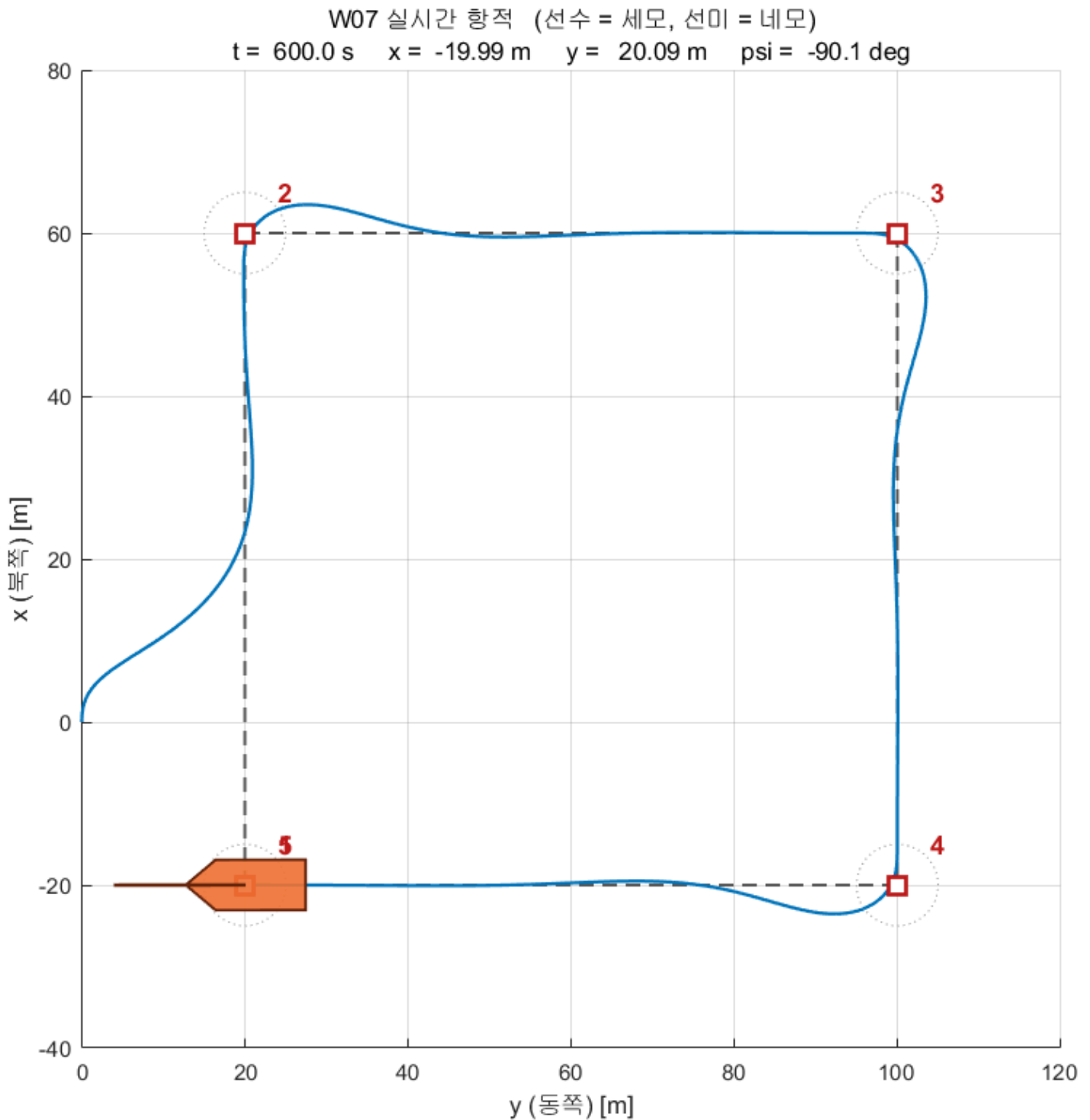
오프라인 모델 실행 중 (auto\_run = 0 으로 두면 실행하지 않는다)  
실시간 그림 켜짐 - 새 창이 열린다. 끄려면 animate = 0

===== LOS 유도 / 조류 0 m/s =====  
평균 |y\_e|                    1.975 m    (전 구간)  
평균 |y\_e|                    0.700 m    (초기 60초 제외) <- 과제에 쓸 값  
최대 |y\_e|                    20.000 m  
완주 시각                    206.5 s  
평균 |크랩각|                2.97 deg  
정상상태 속도                1.500 m/s    (목표 1.50)

• 전체 3초 정도 걸리고 그림 창 3개가 뜸

| 창          | 내용                          |
|------------|-----------------------------|
| W07 실시간 항적 | 도는 동안 배가 움직이는 것을 보여 줌       |
| W07 (성능)   | 궤적 · 이탈량 · 헤딩 · 크랩각 · 속도 5장 |
| W07 추진기    | 추력과 프로펠러 회전수                |

## 실시간 항적 창



- 모델 안의 **Animate** 블록이 매 스텝 `W07_animate.m` 을 불러 그림
- 선체 모양은 `W07_matlab/shipModel.m` 과 같음 — **선수**는 세모, **선미**는 네모
- 빨간 사각형이 웨이포인트, 점선 원이 수락반경 R
- 제목 줄에 현재 시각 · 위치 · 선수각이 계속 갱신됨

| 설정 ( <code>W07_setup.m</code> )   | 뜻                                  |
|-----------------------------------|------------------------------------|
| <code>animate = 1</code>          | 0 으로 두면 그림을 그리지 않아 훨씬 빨리 돌         |
| <code>animate_every = 0.25</code> | 몇 초마다 다시 그릴지. 키우면 빨라짐              |
| <code>boat_scale = 3</code>       | 선체를 몇 배로 크게 그릴지. 1 이면 실제 크기(4.9 m) |

### ! MATLAB Function 블록은 기본적으로 그래픽 출력을 지원하지 않음

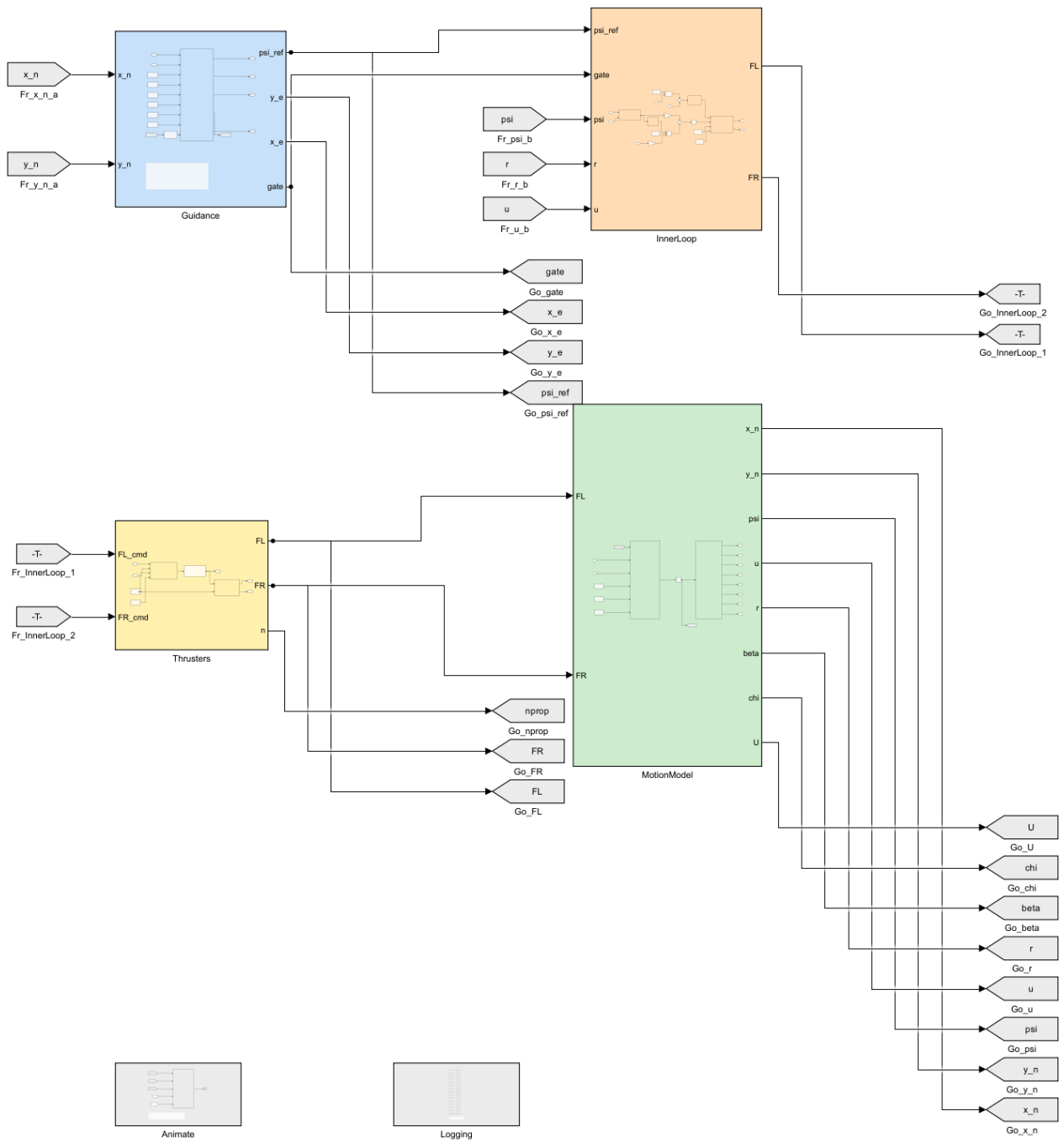
코드 생성 대상이기 때문임. `coder.extrinsic` 으로 선언하면

컴파일하지 않고 MATLAB 함수를 그대로 부름. 그림·파일입출력에 쓰는 표준 방법임

```
coder.extrinsic('W07_animate');
```

## B. 오프라인 모델 — 구조 읽기 (20분)

```
open_system('W07_0_offline')
```



## 왼쪽에서 오른쪽으로 읽는다

| 단 | 블록          | 하는 일                                                                                       |
|---|-------------|--------------------------------------------------------------------------------------------|
| 1 | Guidance    | 위치와 웨이포인트 → <code>psi_ref</code> , <code>y_e</code> , <code>x_e</code> , <code>gate</code> |
| 2 | InnerLoop   | 헤딩 P-D + 속도 PI + 추력배분 → 좌·우 추력 지령                                                          |
| 3 | Thrusters   | 프로펠러·모터 모델 → 실제 추력                                                                         |
| 4 | MotionModel | 운동방정식 + 적분 + 상태 산출                                                                         |
| — | Animate     | 도는 동안 배와 웨이포인트를 그림 (제어에는 관여 안 함)                                                           |

- 되먹임은 **Goto/From** 태그로 보냄. 화면을 가로지르는 선이 없음
- 웨이포인트·게인 같은 **설정값은 각 상자 안에** 있음. 최상위에는 신호만 보임
- `Logging` 과 `Animate` 는 **입출력 포트가 없는 상자**. 안에서 From 태그로 받음
- 그래프용 로깅은 `Logging` 상자 안에 모여 있음 (더블클릭해서 열어 볼 것)

## 블록 색으로 역할을 구분한다

i 7~9주차의 모든 모델이 같은 색 규칙을 따름

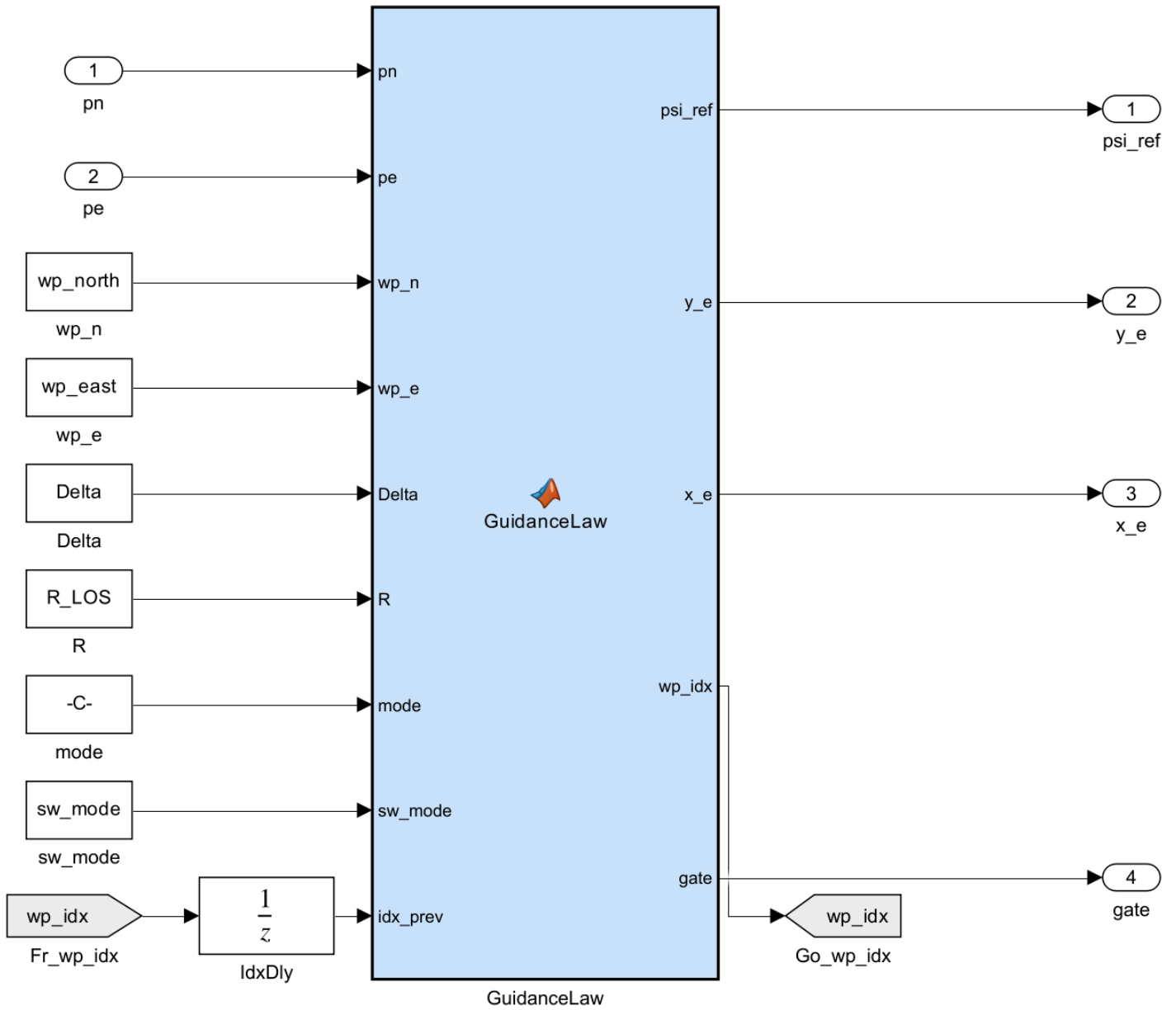
| 색   | 역할            | 예                                                                          |
|-----|---------------|----------------------------------------------------------------------------|
| 파랑  | 유도 (Guidance) | <code>Guidance</code> , <code>WpManager</code> , <code>LoiterVF</code>     |
| 보라  | 미션 판단         | <code>MissionFSM</code> , <code>ModeSwitch</code> , <code>TurnCount</code> |
| 주황  | 제어 (Control)  | <code>InnerLoop</code> , PID, 추력배분                                         |
| 노랑  | 추진기           | <code>Thrusters</code>                                                     |
| 초록  | 운동모델 (Plant)  | <code>MotionModel</code>                                                   |
| 연보라 | ROS 통신        | <code>PoseSubscriber</code> , <code>CmdPublisher</code>                    |
| 회색  | 로깅·표시         | <code>Logging</code> , <code>Animate</code> , Goto/From 태그                 |
| 흰색  | 설정값           | Constant                                                                   |

- 선은 **직선 또는 직각만** 씀. 대각선 없음
- 되먹임처럼 뒤로 돌아가는 선만 꺾임
- 배치를 흐트러뜨렸으면 아래 한 줄로 되돌림 (생성 스크립트가 배치까지 다시 만듦)

```
w07_setup
build_w07_models
```

## Guidance 블록

- 이 블록 하나가 §1-5 부터 §1-8 까지를 전부 담고 있음. **§1-9 의 표를 옆에 놓고 열어 볼 것**
- 출력은 네 개임 — `psi_ref` (지령), `y_e` (이탈량), `x_e` (경로 방향 오차), `gate` (미션 종료)



유도 — 위치를 받아 목표 선수각을 낸다.  
 mode = 1 atan2 : 다음 웨이포인트를 곧장 겨눈다  
 mode = 2 LOS : 두 웨이포인트를 잇는 직선을 따라간다

전환 판정 sw\_mode  
 1 경로 방향 :  $d - x_e < R$  (MSS 기본값. 밀려나도 멈추지 않는다)  
 2 수락 원 : 다음 웨이포인트까지 직선거리  $\leq R$

wp\_idx 는 밖으로 나가지 않는다. 지금 몇 번째 구간을 달리는지를 유도부가 스스로 기억하는 값이라, 한 스텝 지연을 거쳐 자기에게 되돌아온다. 이 지연이 없으면 대수 루프가 되어 모델이 컴파일되지 않는다.

두 오차 — 두 유도법칙이 같은 식을 씀

```

pi_p = atan2(yk1 - yk, xk1 - xk); % 경로 방위각
x_e = (x_n - xk)*cos(pi_p) + (y_n - yk)*sin(pi_p); % 경로 방향 오차
y_e = -(x_n - xk)*sin(pi_p) + (y_n - yk)*cos(pi_p); % 경로 이탈량

```

- $y_e$  를 두 모드 모두 같은 식으로 계산함 → 그래야 같은 잣대로 비교할 수 있음

#### 유도법칙 — **mode** 로 고름

```

if mode == 1
    psi_ref = atan2(yk1 - y_n, xk1 - x_n); % atan2 유도
else
    psi_ref = pi_p - atan(y_e / Delta); % LOS 유도
end

```

#### 전환 판정 — **sw\_mode** 로 고름

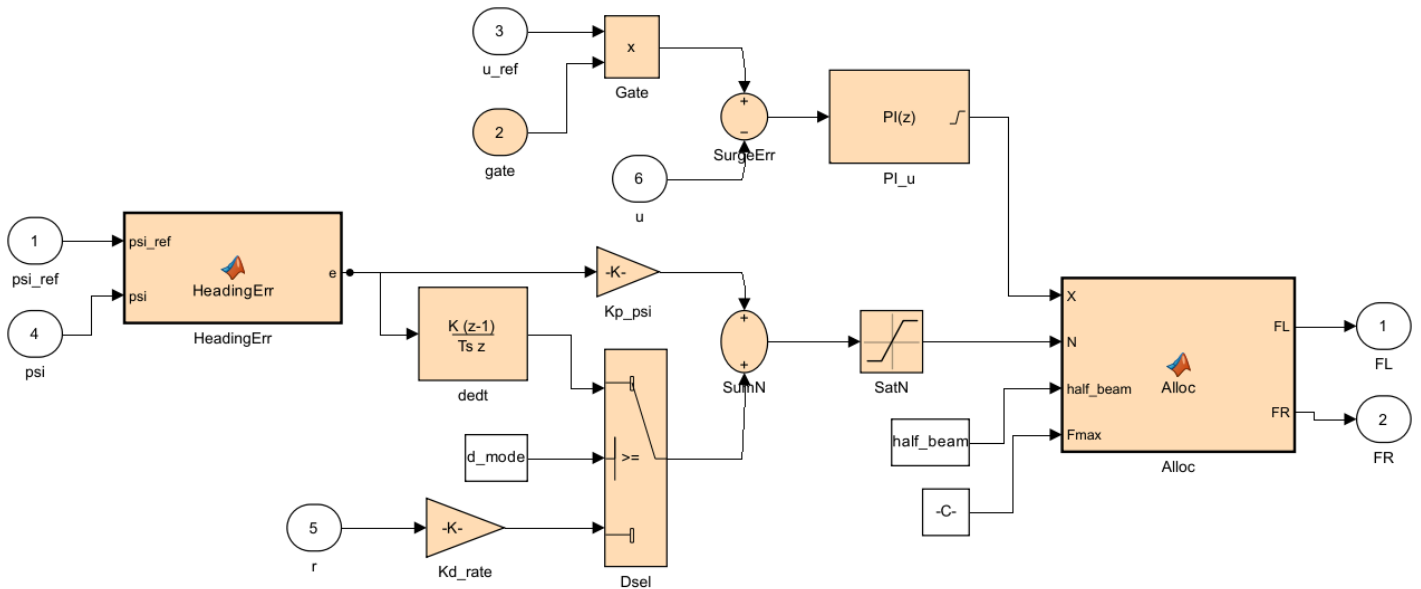
```

d = sqrt((xk1 - xk)^2 + (yk1 - yk)^2); % 구간 길이
if sw_mode == 1
    reached = (d - x_e) < R; % 경로 방향 (기본값)
else
    reached = sqrt((xk1 - x_n)^2 + (yk1 - y_n)^2) <= R; % 수락 원
end

```

- 웨이포인트 번호는 **Unit Delay** 로 되먹임함 (한 스텝 전 값을 기억). 이 지연이 없으면 대수 루프가 되어 컴파일 되지 않음

#### InnerLoop 서브시스템



#### 헤딩 제어 — P-D 요각속도 되먹임

$$\tau_N = K_{p,\psi} ssa(\psi_{ref} - \psi) - K_{d,\psi} r$$

### ★ 오차를 미분하지 않음

- 6주차 H절의 `HeadingCtrl` 과 같은 식임. 여기서는 지령이 계속 바뀜
- 웨이포인트가 바뀌는 순간 `psi_ref` 가 갱충 됨 → 오차 미분값이 폭발함 (미분 킅)
- 대신 **요각속도  $r$  을 직접 되먹임함**.  $r$  은 튀지 않음
- MSS 툴박스의 Otter 데모가 쓰는 방식이 이것임

- 두 방식을 `d_mode` 로 골라 비교할 수 있게 해 두었음 (`Dsel` 스위치)

| <code>d_mode</code> | D 항            | 특징                         |
|---------------------|----------------|----------------------------|
| 1 (기본)              | $-K_d r$       | 요각속도 되먹임. 지령이 튀어도 D 항은 안 튼 |
| 2                   | $+K_d \dot{e}$ | 오차 미분. 웨이포인트 전환 때 추력이 크게 튼 |

- 기준 환경 실측값 — 한 스텝당 추력 변화 최대값

| <code>d_mode</code> | $T_s = 0.05$ | $T_s = 0.20$ |
|---------------------|--------------|--------------|
| 1 요각속도              | 51 N         | 175 N        |
| 2 오차 미분             | 85 N         | 232 N        |

### i 추종 성능은 둘이 비슷함

차이는 **추력 명령이 얼마나 거칠어지는가**에서 남  
실제 추진기에는 이 거친 명령이 그대로 부담이 됨

### 속도 제어 — surge velocity 되먹임 PI

$$\tau_X = \text{PI}(u_{\text{ref}} \cdot \text{gate} - u)$$

### ⚠ $U = \sqrt{u^2 + v^2}$ 가 아니라 $u$ 를 되먹임함

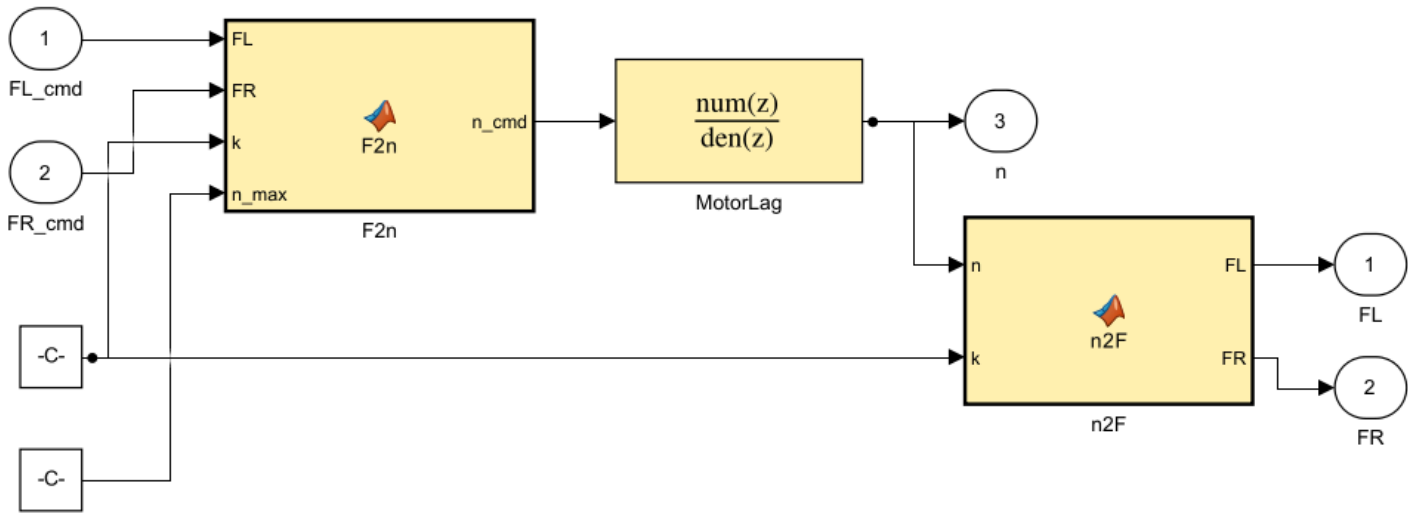
- $u, v$  모두 대지 기준 속도의 선체 성분임 (1-2절).  $U$  는 그 크기(대지속력)라 옆으로 밀리는 성분  $v$  까지 포함함
- 추진기가 만드는 힘은 **전후 방향**이므로  $u$  로 제어하는 것이 맞음
- MSS 의 `surge velocity control` 서브시스템 입력도 `u_d` 와  $u$  임

### 추력배분 — 4주차의 차동 배분 그대로

$$F_L = \frac{X}{2} + \frac{N}{2b}, \quad F_R = \frac{X}{2} - \frac{N}{2b}, \quad b = 1.027 \text{ m}$$

- $b$  = 후방 두 추진기의 좌우 **반폭** (`w07_setup` 의 `half_beam`)

## Thrusters 서브시스템



### ! Gazebo 는 추력을 즉시 냄. 실제 모터는 그렇지 않음

VRX 의 추진기 플러그인은 뉴턴 단위 추력을 받아 곧바로 프로펠러 속도를 지정함  
지연도, 회전수 한계도 없음. 그래서 제어기 쪽에 프로펠러·모터 모델을 붙임

|                                                                          |           |
|--------------------------------------------------------------------------|-----------|
| $F_{cmd} \rightarrow n_{cmd} = \text{sign}(F) \cdot \sqrt{ F /k_{prop}}$ | 역 프로펠러 법칙 |
| $\rightarrow  n  \leq n_{max}$                                           | 회전수 포화    |
| $\rightarrow 1차 지연 (시상수 \tau_n)$                                         | 모터 응답     |
| $\rightarrow F = k_{prop} \cdot n \cdot  n $                             | 정 프로펠러 법칙 |

### ⚠ 같은 글자, 다른 뜻 — $k$ 와 $n$

| 글자  | 1부 · §1-9 (유도)     | 이 절 (추진기)                                                   |
|-----|--------------------|-------------------------------------------------------------|
| $k$ | 웨이포인트 번호           | 프로펠러 추력 계수 → 혼동을 피해 $k_{prop}$ 으로 씀 ( $w07\_setup$ 과 같은 이름) |
| $n$ | 웨이포인트 개수 (§1-9 코드) | 프로펠러 회전수 [rad/s]                                            |

- $k_{prop} = \rho C_t D^4 = 1000 \times 0.004422 \times 0.2^4 = 0.0070752$   
— VRX 플러그인이 내부에서 쓰는 값과 같은 값
- $n_{max} = \sqrt{F_{max}/k_{prop}} = 266 \text{ rad/s}$  (  $F_{max} = 500$  기준, 약 2540 rpm)
- 포화가 힘이 아니라 회전수에 걸린다는 점이 중요함. 추력은 회전수의 제공이므로 둘은 다름

★ **F\_max** 를 왜 500 N 으로 두는가

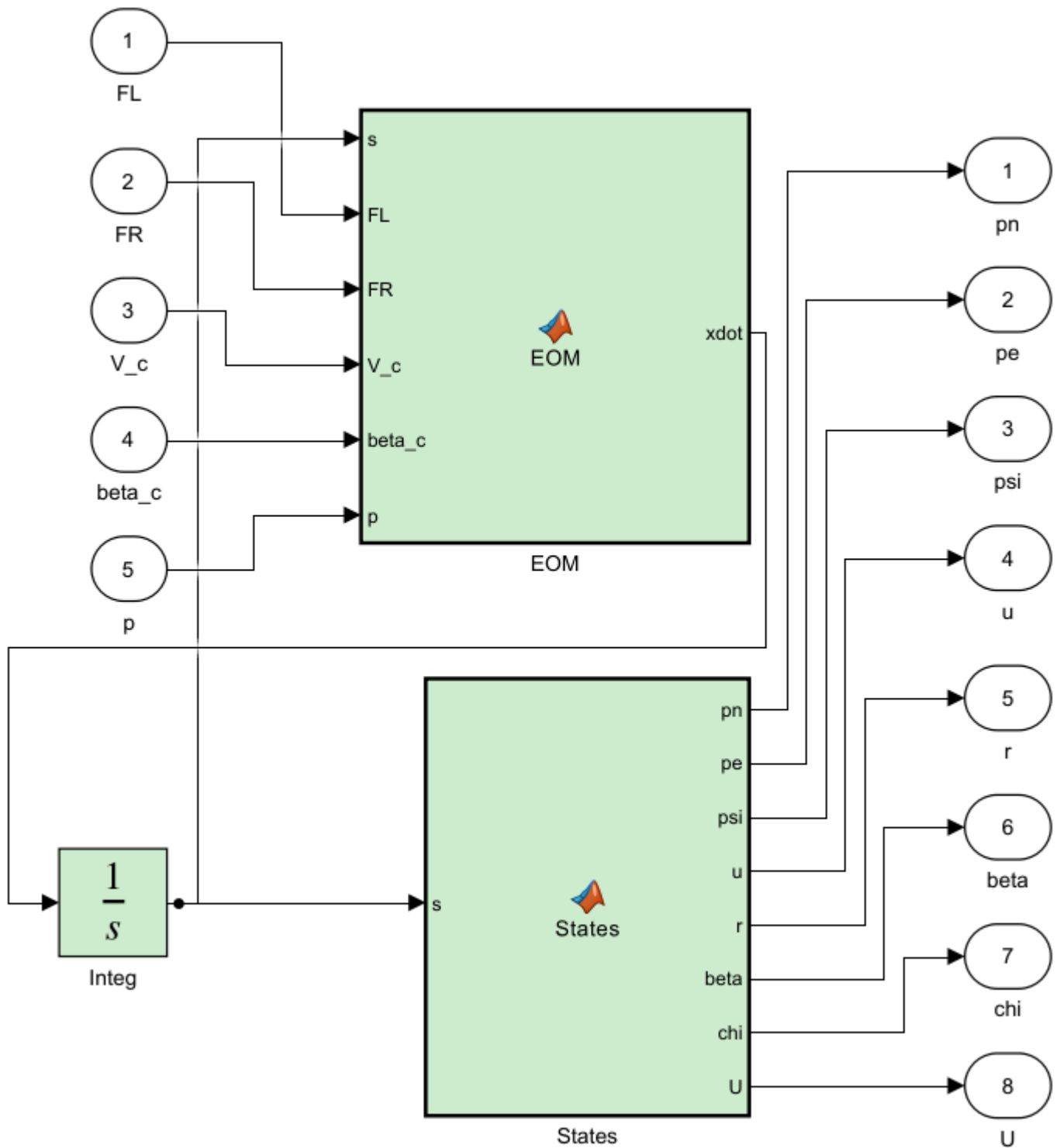
1.5 m/s 를 유지하는 데만  $X = (100 + 150 \times 1.5) \times 1.5 = 488$  N, 편당 244 N 이 듬  
6주차처럼 250 N 으로 제한하면 직진에 다 써 버려 선회할 여유가 없음

| <b>F_max</b> | 포화 시간 비율   | 정착 후 평균  y_e   |
|--------------|------------|----------------|
| 250 N        | 77.1 %     | 1.341 m        |
| 400 N        | 2.6 %      | 0.728 m        |
| <b>500 N</b> | <b>0 %</b> | <b>0.700 m</b> |

VRX 플러그인의 물리 한계는 2353.6 N 이므로 500 은 충분히 현실적임

**포화는 게인 문제로 위장함** — 게인을 아무리 만져도 나아지지 않으면 포화부터 확인할 것.

## MotionModel 서브시스템



- 운동방정식 → 연속 적분기 → 상태 산출을 한 블록에 담음
- VRX 모델에서 **Gazebo** 가 차지하는 자리와 정확히 같음

★ 이 운동모델은 **Gazebo** 와 같은 계수를 씀

임의로 만든 모델이 아님. VRX 플러그인 설정에서 그대로 가져온 값임

| 항목    | 값                      | 출처                                                           |
|-------|------------------------|--------------------------------------------------------------|
| 질량    | 211 kg                 | <code>wamv_base.urdf.xacro</code> 180 + 엔진 2×15 + 프로펠러 2×0.5 |
| 요 관성  | 653 kg·m <sup>2</sup>  | 위 파일의 <code>izz</code> 446 + 엔진 이동항                          |
| 전후 항력 | $-(100 + 150 \cdot u)$ | $u$                                                          |
| 좌우 항력 | $-(100 + 100 \cdot v)$ | $v$                                                          |
| 요 항력  | $-(800 + 800 \cdot r)$ | $r$                                                          |
| 부가질량  | 0                      | <code>xDotU = yDotV = nDotR = 0</code>                       |

- 계산: 추력 400 N 에서  $400 = (100 + 150u)u \rightarrow u = 1.333 \text{ m/s}$
- 6주차에 Gazebo 로 실측한 값은  $1.332 \text{ m/s}$  였음 (6주차 J절)

### i 조류는 오프라인 모델에만 존재함

VRX 2.4.0-2 에는 해류 플러그인이 없음. 레포 전체를 검색해도 나오지 않고,

`SimpleHydrodynamics` 가 항력을 대지속도로 계산하므로 구조적으로 해류가 존재할 수 없음

그래서 조류 실험은 이 오프라인 모델에서만 할 수 있음

구현은 간단함 — 항력을 물에 대한 상대속도로 계산하면 됨

```
u_r = u - V_c · cos(beta_c - psi)
v_r = v - V_c · sin(beta_c - psi)
```

- $V_c$  = 조류 속도 ( `current_speed` )
- $\beta_c$  = 조류가 흘러가는 방위, 북에서 시계방향 ( `current_direction` 을 라디안으로 바꾼 값)
- $u, v$  는 대지 기준 속도의 선체 성분이므로, 물에 대한 상대속도  $u_r, v_r$  을 따로 만들어 항력에 넣음

### ⚠ 같은 글자, 다른 뜻 — $\beta$ 와 $\beta_c$

| 기호        | 뜻          | 기준                            |
|-----------|------------|-------------------------------|
| $\beta$   | 크랩각 (1-2절) | 뱃머리 $\psi$ 에서 진행 방향 $\chi$ 까지 |
| $\beta_c$ | 조류 방향      | 북에서 조류가 흘러가는 방향까지             |

## C. 실습 1 — atan2 vs LOS (25분)

### C-1. LOS 로 한 번

`w07_setup`

- 실시간 항적 창에서 배가 20 m 벗어난 곳에서 출발해 경로에 붙는 것을 눈으로 확인
- 이어서 성능 그림 5장 · 추진기 그림 2장과 지표가 나옴

### C-2. atan2 로 바꿔서

- `w07_setup.m` 에서 `guidance_mode = 1` 로 고치고 다시 실행

`w07_setup`

- 두 궤적의 모양이 어떻게 다른지 비교할 것

C-3. 네 조건을 한 번에

W07\_compare

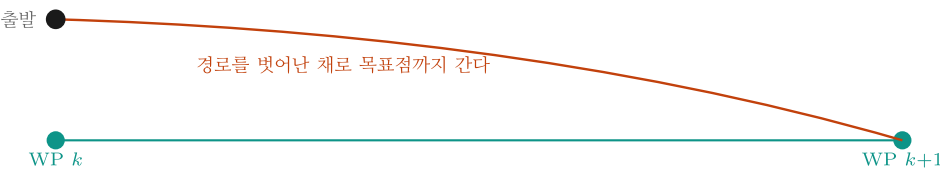
정상 출력

| 조건             | mean ye | 정착후   | max ye | 완주[s] | 크랩[deg] |
|----------------|---------|-------|--------|-------|---------|
| atan2 / 조류 없음  | 3.934   | 1.757 | 20.000 | 201.5 | 1.87    |
| LOS / 조류 없음    | 1.975   | 0.700 | 20.000 | 206.5 | 2.97    |
| atan2 / 조류 0.4 | 4.734   | 3.850 | 20.000 | 202.7 | 8.08    |
| LOS / 조류 0.4   | 2.673   | 1.504 | 20.000 | 207.0 | 8.04    |

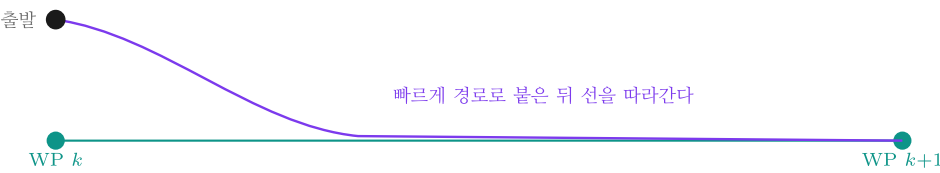
정착후 = 초기 수렴 60 초를 뺀 평균. 유도법칙 비교에는 이 값을 쓴다

atan2 는 LOS 보다 2.51 배 (조류 없음), 2.56 배 (조류 있음) 더 벗어난다

atan2 유도      다음 웨이포인트를 곧장 겨냥



LOS 유도      경로 위 Δ 앞의 한 점을 겨냥



오프라인 WAM-V 실험 — 평균 경로 이탈량 (초기 수렴 60초 제외)

| 조건    | atan2  | LOS    | 배율    |
|-------|--------|--------|-------|
| 조류 없음 | 1.76 m | 0.70 m | 2.5 배 |

관찰할 것

★ 다음 세 가지를 확인함

- 1. 조류가 없어도 LOS 가 2.5 배 정확함 — 유도법칙 자체의 차이
- 2. 조류를 켜면 두 법칙 모두 나빠지고 벌어지는 거리가 커짐 — 정착 후 차이가 1.06 m 에서 2.35 m 로. 비율은 2.5 배 그대로임
- 3. 크랩각이 3° → 8.0° 로 커짐 (조류 0.4 m/s) — 뱃머리와 진행 방향이 8도 어긋남

### ! 비율만 보면 농침

atan2 는 1.76 m → 3.85 m 로 **2.1 m** 나빠졌고, LOS 는 0.70 m → 1.50 m 로 **0.8 m** 나빠졌음  
조류가 같은 만큼 밀었는데 atan2 가 2.6 배 더 밀렸음. 비율이 같게 나오는 것은  
둘 다 나빠졌기 때문임. **비교할 때는 비율과 차이를 함께 봄**

### ! LOS 도 조류에서 오차가 남음

조류 없음 0.70 m → 조류 있음 1.72 m. 완전히 0 이 되지는 않음  
선수각  $\psi$  를 제어하기 때문임 (1-2절). 이 잔류 오차를 없애는 것이  
**적분 LOS(ILOS)** 임. 관심 있는 팀은 Term Project 에서 시도해 볼 것

## D. 실습 2 — $\Delta \cdot R$ · 전환 판정 바꿔 보기 (20분)

### D-1. lookahead distance

- 먼저 §1-7 의 예제를 그대로 돌려 봄. 네 조건을 알아서 돌리고 그림과 표를 냄

```
>> W07_lookahead_demo
```

- 그다음 `W07_setup.m` 에서 `Delta` 만 바꾸며 미션 전체를 실행함. 나머지는 손대지 않음

```
Delta = 2; % 2 · 3 · 5 · 10 · 20 · 30 으로 여섯 번
```

- 각 조건에서 `W07_plot` 이 찍는 **정착 후 평균  $|y_e|$**  와 **완주 시각**을 적음
- 궤적 그림에서 **경로를 몇 번 가로지르는지도** 눈으로 셈

| 볼 것         | $\Delta$ 가 작을 때 | $\Delta$ 가 클 때 |
|-------------|-----------------|----------------|
| 경로에 붙는 속도   | 빠름              | 느림             |
| 궤적 모양       | 지그재그            | 완만             |
| 요 모멘트를 쓰는 양 | 많음              | 적음             |

- §1-7 의 표와 맞는지 대조할 것.** 조류 없이 `Delta = 10` 이면 정착 후 평균이 **0.700 m** 여야 함
- 그다음 `current_speed = 0.4`, `current_direction = 90` 으로 두고 다시 여섯 번 돌림. **최적  $\Delta$  가 어디로 옮겨 가는지** 봄 (1-7절 두 번째 표와 같은 조건)

### D-2. 수락반경

- 이번에는 `R_LOS` 만 바꿈

```
R_LOS = 2; % 2 · 5 · 10 · 20
```

| R       | 볼 것                    |
|---------|------------------------|
| 2       | 모서리를 정확히 둠. 회전이 급함     |
| 5       | 기본값                    |
| 10 · 20 | 모서리를 크게 잘라먹고 그만큼 일찍 끝남 |

- 궤적 그림에서 배가 웨이포인트에 **얼마나 가까이 갔는지**를 봄 (§1-8 의 "모서리 컷")

### D-3. 전환 판정 — 수락 원은 걸릴 수 있다

- §1-8 에서 계산으로 보인 것을 직접 걸리게 만들어 봄

```
R_LOS          = 2;      % 수락 원을 좁게
current_speed   = 0.4;    % 조류로 옆으로 밀어낸다
current_direction = 90;
sw_mode         = 2;      % 수락 원 판정
```

- 실행하면 배가 **WP2 (60, 20)** 옆을 스쳐 지나간 뒤 전환하지 못하고, 첫 구간의 연장선을 따라 계속 북쪽으로 감. 미션이 끝나지 않음
  - LOS 지령에는  $x_e$  가 없어서 지나친 웨이포인트로 되돌아가지 않음 (§1-8)
- `sw_mode = 1` 로만 바꿔 다시 돌림. 같은 조건에서 정상적으로 완주함

| sw_mode   | 결과         | 전환 횟수 |
|-----------|------------|-------|
| 2 (수락 원)  | 미완주        | 0     |
| 1 (경로 방향) | 214.1 s 완주 | 3     |

- 왜 그런지를 §1-8 의 부등식으로 설명할 수 있어야 함

 Scope 대신 w07\_plot 사용을 권장함

궤적·이탈량·헤딩·크랩각·속도·추력을 한 번에 그려 주고  
성능 지표를 표로 찍어 줌

## E. 실습 3 — VRX 연동 (50분)

### E-1. ground truth odometry 켜기

- 6주차 A절과 같은 방법. 이미 해 두었으면 건너뛴

```
mkdir -p ~/capstone_ws/wamv
cp ~/vrx_ws/src/vrx/vrx_urdf/wamv_gazebo/urdf/wamv_gazebo.urdf.xacro \
  ~/capstone_ws/wamv/w7_wamv.urdf.xacro
```

- `ground_truth_enabled` 의 `default` 를 `true` 로 수정

```
nano ~/capstone_ws/wamv/w7_wamv.urdf.xacro
```

### E-2. VRX 실행

```
ros2 launch vrx_gz competition.launch.py \
  world:=sydney_regatta \
  urdf:=$HOME/capstone_ws/wamv/w7_wamv.urdf.xacro
```

⚠ 3주차에서 RTF 가 1 % 미만이었던 노트북은 옵션을 함께 줌

```
ros2 launch vrx_gz competition.launch.py world:=sydney_regatta urdf:=$HOME/capstone_ws/wamv/w7_wamv.urdf.xacro "extra_gz_args:---render-engine-server ogre"
```

### ★ 실험마다 VRX 를 다시 띄움

앞 실험이 끝난 자리(경로 끝)에서 다음 실험을 시작하면 결과가 완전히 달라짐

- 모든 실험은 스폰 위치 · 초기 선수각이 같은 상태에서 시작하는 것을 전제로 함
- 8주차 로이더링 실습도 같음 — 이어 돌리면 원에 도달하지 못함

## E-3. 실시간 계수(RTF) 측정 — 반드시 할 것

### ✕ 이 단계를 건너뛰면 결과가 틀림

Gazebo 는 컴퓨터에 따라 실시간보다 느리게 돌

Simulink 를 벽시계에 맞춰 돌리면 배는 그만큼 적은 시뮬레이션 시간을 살게 됨

```
ros2 topic echo /wamv/sensors/position/ground_truth_odometry --once | grep -m1 "sec:"
```

- 20초 기다렸다가 다시 실행해서 **시뮬레이션 시각이 몇 초 늘었는지** 확인

20초 기다렸는데 시뮬 시각은 10초만 늘었다 →  $RTF = 0.5$

- 한 줄로 보려면 (WSL)

```
gz topic -e -t /stats -n 1 | grep real_time_factor
```

| 환경                                  | RTF                               |
|-------------------------------------|-----------------------------------|
| 기준 환경 (2026-09-04)                  | <b>0.48</b>                       |
| 기준 노트북 (2026-09-15, <b>ogre</b> 옵션) | <b>0.83 ~ 0.98</b> — 같은 월드에서도 흔들림 |

## E-4. 모델 실행 — 한 줄이면 된다

```
setappdata(0, 'W07_skip_run', true); % W07_setup 의 오프라인 자동 실행을 건너뛸  
W07_setup  
W07_vrx_run
```

- **W07\_vrx\_run.m** 이 아래를 대신 해 줌

| 단계 | 하는 일                                                                  |
|----|-----------------------------------------------------------------------|
| 0  | Simulink 프로파일과 MATLAB 의 ROS 도메인을 맞춤                                   |
| 1  | 토픽이 보이는지 확인 (안 보이면 <b>여기서 멈추고 이유를 알려 줌</b> )                          |
| 2  | <b>RTF 를 20초 동안 직접 측정</b> — 손으로 재지 않아도 됨. 이때 <b>배의 현재 위치를 원점으로</b> 잡음 |
| 3  | 페이싱 비율을 RTF 에 맞추고 실행                                                  |
| 4  | 끝나면 오프라인과 <b>같은 형식의 결과 그림</b>                                         |
| 5  | 같은 초기 선수각으로 오프라인을 돌려 <b>두 궤적을 겹쳐 그림</b> . 오프라인 지표도 같은 형식으로 찍음         |

- 기본 230 s — 임무(약 207 s)를 끝까지 봄. **W07\_vrx\_run(150)** 이면 완주 전에 끊김. 대조를 건너뛰려면 **W07\_vrx\_run(230, false)**

### ✕ 스폰 위치는 설치마다 다름 — 스크립트가 직접 잼

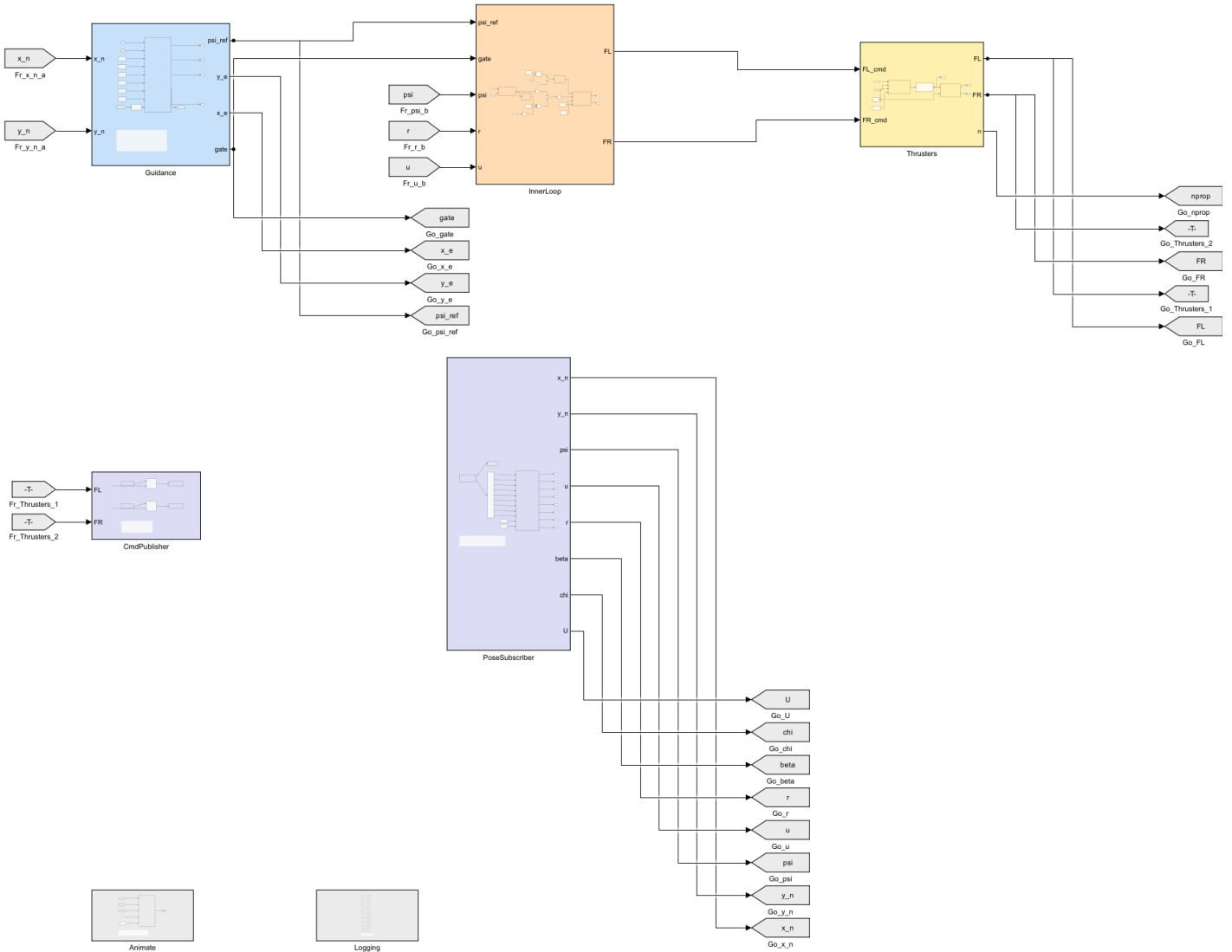
- `competition.launch.py` 의 `Model('wamv', ...)` 스폰 좌표가 같은 VRX 2.4.0-2 인데도 컴퓨터마다 달랐음:  $y_{ENU} = 162$  와  $200$
- `W07_setup.m` 의 `origin_north = 162` 는 기본값. `W07_vrx_run` 이 ground truth 첫 샘플로 덮어씀 → 2 단계 출력의 스폰 위치  $(x, y) = \dots$
- 고정값이 틀리면 경로 전체가 그 차이만큼 밀린 채 달림 (9주차 D절에서 38 m 밀려 육지에 걸린 실측이 있음)

### 🔍 주행 중 실시간 항적이 표시됨

VRX 모델에도 오프라인과 같은 `Animate` 블록이 들어 있음

배가 어디쯤 가고 있는지 Gazebo 화면 없이도 보임. 그러면 `animate = 0`

W07 실습 E — VRX 연동 (Gazebo 필요)  
 Guidance -> Inner Loop -> Thrusters -> Navigation  
 위도부터 내부루트로 추진기>제어인 W07\_0\_offline 과 연결이 됩니다.  
 문동모달만 Gazebo 로 바뀝니다.  
 실행 전: (1) W07\_setup (2) VRX 기동 (3) 제어실용 RTF 에 맞출 것



- 손으로 하고 싶으면 (수업에서 한 번은 해 볼 것). 비율은 E-3 에서 팬 RTF, 원점은 스폰 좌표

```
origin_north = 200;   origin_east = -532;    % 자기 launch 파일의 스폰 좌표 (ENU y, x)
set_param('W07_1_vrx','EnablePacing','on','PacingRate','0.70','StopTime','230');
out = sim('W07_1_vrx');
W07_plot(out, 'VRX / LOS')
```

1. Run 옆 화살표 → **Simulation Pacing** → Enable 체크
2. **비율을 RTF 값으로** 설정 ← 6주차와 다른 점
3. 정지 시간 **230** 으로 두고 Run

## E-5. 결과 확인

- 기준 환경 실측값 (2026-09-19, 데스크톱, VRX 2.4.0-2, 스폰 ENU (**-532, 200**), VRX 새로 띄운 뒤 실행)

```

0) ROS_DOMAIN_ID = 8
1) VRX 토픽 확인
   OK - 토픽 39개
2) RTF 측정 (20초)
   스폰 위치 (x, y) = (200.0, -532.0) m - 이 점을 원점으로 쓴다
   RTF = 0.681 -> 페이싱 비율을 이 값으로 둔다
3) W07_1_vrx 실행 (230초)
4) 결과 그림

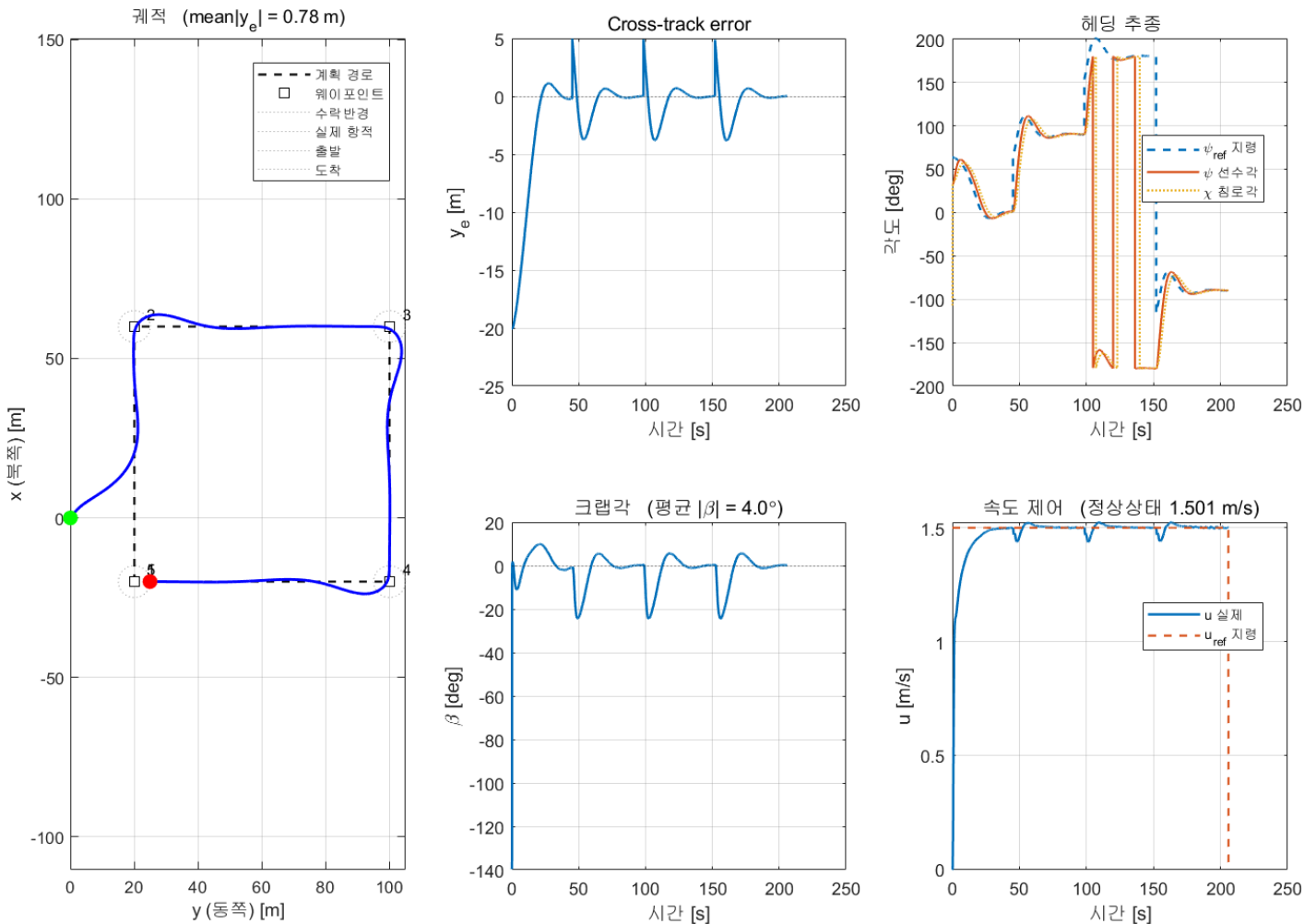
===== VRX / LOS 유도 =====
평균 |y_e|           1.906 m   (전 구간)
평균 |y_e|           0.784 m   (초기 60초 제외) <- 과제에 쓸 값
최대 |y_e|           20.049 m
완주 시각            205.9 s
평균 |크랩각|         4.01 deg
정상상태 속도        1.501 m/s (목표 1.50)
그림 저장: img/W07_1_vrx_result.png
5) 같은 초기 선수각으로 오프라인 모델 실행
   VRX 초기 선수각 32.82 deg
  
```

- 이어서 오프라인 지표 6줄과 대조 결과(E-6)가 출력됨
- 스크립트가 낸 RTF(0.681)는 Gazebo 가 표시하는 RTF(09-18 데스크톱에서 0.99)보다 낮음. MATLAB 쪽에서 메시지 도착 간격으로 재기 때문. **이 값을 그대로 페이싱에 씀** — 8주차 D-2 경고 참고

| 읽는 법                 |                                             |
|----------------------|---------------------------------------------|
| 완주 시각 205.9 s        | 오프라인 206.7 s 와 0.8 s 차이                     |
| 정착 후 $ y_e $ 0.784 m | 오프라인 0.702 m. 유도 성능은 시뮬레이터가 바뀌어도 같음         |
| 정상상태 속도 1.501 m/s    | 추력·항력 계수가 같다는 증거                            |
| 최대 $ y_e $ 20 m      | 첫 웨이포인트가 출발선에서 20 m 떨어져 있어서 나오는 값           |
| 평균 크랩각 4.01°         | 오프라인 2.97° 보다 큼. VRX 선체는 선회 중 옆으로 조금 더 미끄러짐 |

### i 토픽 개수는 VRX 판·센서 설정에 따라 다름

같은 2.4.0-2 + ground truth 에서 **37개** 와 **39개** 를 모두 봤음. 개수 자체보다 **필요한 토픽이 있는지**가 중요함



★ 주행거리 ÷ 시간 ≈ **u** 평균이 되어야 함

이 둘이 2배쯤 차이 나면 **페이싱 비율이 RTF 와 맞지 않은 것임**

⚠ 첫 샘플은 사용하지 않음

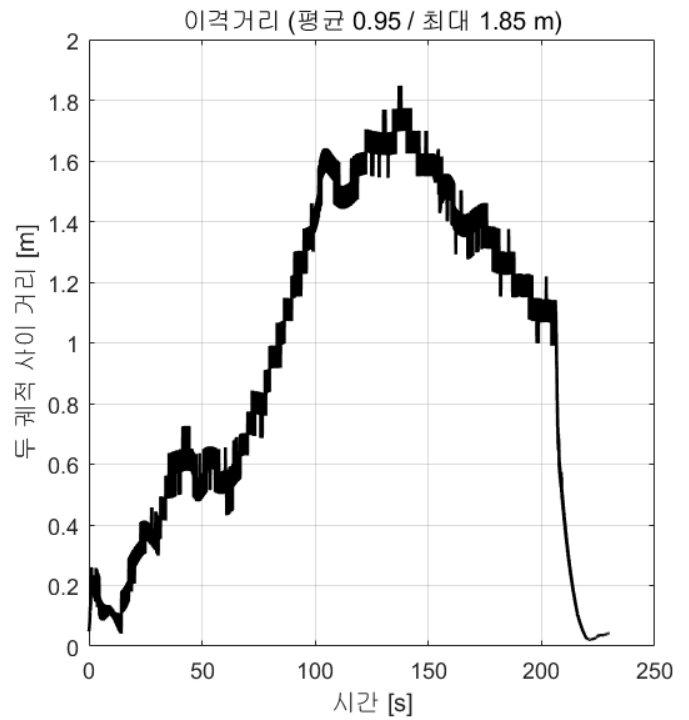
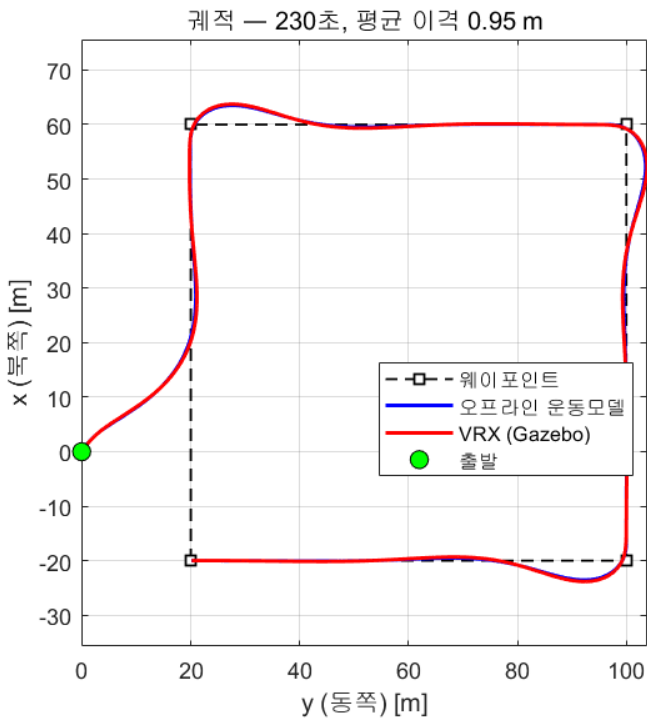
Subscribe 블록은 첫 메시지가 도착하기 전에는 0 으로 채운 버스를 내보냄

그대로 좌표 변환에 넣으면 위치가 스폰 원점만큼(약 570 m) 튼

**PoseSubscriber** 서브시스템 안의 **Nav** 함수 첫머리에 **if ex == 0 && ey == 0** 가드가 들어 있는 이유임  
구독한 값을 쓰기 전에 유효한지 확인하는 것 — ROS 를 쓰는 내내 필요한 습관임

## E-6. 오프라인 모델과 비교 — 본 절의 결론

- **w07\_vrx\_run** 이 자동으로 그려 줌



- 기준 환경 실측값 (2026-09-19, 230 s, 같은 초기 선수각 32.82°)

| 항목                    | 오프라인         | VRX          | 차이            |
|-----------------------|--------------|--------------|---------------|
| 주행거리 [m]              | 311.9        | 312.8        | +0.9          |
| 완주 시각 [s]             | 206.7        | 205.9        | -0.8          |
| 60초 이후 평균 $ y_e $ [m] | 0.702        | 0.784        | +0.082        |
| 정상상태 $u$ [m/s]        | <b>1.500</b> | <b>1.501</b> | <b>+0.001</b> |
| 궤적 평균 이격 [m]          |              |              | 0.95          |
| 궤적 최대 이격 [m]          |              |              | 1.85          |

- 같은 설정을 VRX 를 새로 띄워 세 번 돌린 결과 — 이격만 크게 달라짐

| 실행                  | 측정 RTF | 궤적 평균 이격 [m] | 정착 후 $ y_e $ [m] | 정상상태 $u$ [m/s] |
|---------------------|--------|--------------|------------------|----------------|
| 2026-09-18          | 0.697  | 0.28         | 0.785            | 1.501          |
| 2026-09-19 1회       | —      | 2.59         | 0.779            | 1.501          |
| 2026-09-19 2회 (위 표) | 0.681  | 0.95         | 0.784            | 1.501          |
| 오프라인                | —      | —            | 0.702            | 1.500          |

★ 이격 값 하나가 아니라, 정착 후  $|y_e|$  (0.784 vs 0.702) 와 정상상태 속도 (1.501 vs 1.500) 가 같다는 것이 결론

- 궤적 평균 이격은 0.28 · 2.59 · 0.95 m 로 실행마다 크게 달라짐. 페이싱 비율(측정 RTF)과 실제 진행이 어긋난 만큼 같은 길을 다른 시각에 지나기 때문 → 위상 오차
- 정착 후  $|y_e|$  는 0.779 ~ 0.785 m, 정상상태  $u$  는 세 번 모두 1.501 m/s — 계인을 하나도 바꾸지 않았음
- 오프라인에서 맞춘 것이 실제 시뮬레이터에서도 통한다는 뜻
- 이것이 이번 주차 실습의 결론임 → [오프라인-모델이-시뮬레이터와-같아야-계인이-옳겨간다](#)

### ⚠ 이격이 몇 m 로 커지면 먼저 두 가지를 의심함

- **원점**: 2단계의 **스폰 위치** 가 launch 파일과 다르면 경로 전체가 밀림. 궤적 그림에서 두 선이 **평행하게 떨어져** 있음
- **페이싱**: 비율이 실제 진행과 어긋나면 같은 길을 다른 시각에 지남. 두 선은 **겹치는데** 이격이 시간에 **직선으로** 커짐 → 위상(시간) 오차
- 판정 기준은 이격이 아니라 **정착 후  $|y_e|$  와 정상상태 속도임** → [궤적이-겹치면-이격은-오차가-아니라-자연이다](#)

## F. 모델을 다시 만들어야 할 때

```
w07_setup  
build_w07_models
```

- 두 모델이 새로 생성됨
- 스크립트를 읽으면 각 블록이 어떻게 만들어지는지 알 수 있음

# 마무리

## 이번 주차 요약

| 순서  | 한 일                              | 확인 방법                                                     |
|-----|----------------------------------|-----------------------------------------------------------|
| 1   | 설정 파일 이해                         | <code>w07_setup</code> 출력                                 |
| 2   | 오프라인 모델 구조 읽기                    | 4단 GNC 순서 설명 가능                                           |
| 3   | atan2 vs LOS 비교                  | <code>w07_compare</code> 표                                |
| 4   | 조류 실험                            | 크랩각 $2.97^\circ \rightarrow 8.04^\circ$ (LOS, 조류 0.4 m/s) |
| 5   | $\Delta \cdot R \cdot$ 전환 판정 스위치 | $\Delta = 10 \text{ m}$ 에서 정착 후 $ y_e $ 최저 0.700 m        |
| 5-2 | 수락 원 판정 걸리게 하기                   | $R = 2 \text{ m} \cdot$ 조류 0.4 m/s 에서 미완주                 |
| 6   | VRX 연동                           | 20 m 벗어난 곳에서 경로 복귀                                        |
| 7   | 두 모델 대조                          | 정착 후 $ y_e $ 차이 약 0.08 m, 세 번 실행 모두 같음 (이격은 페이싱에 좌우됨)     |

## 수업 진도 체크

### ★ 다음 주 수업을 따라가기 위한 최소 조건

### 이론 이해

- ☐ 유도 · 항법 · 제어를 각각 한 문장으로 구분해 말할 수 있음
- ☐ 선수각  $\psi$ , 침로각  $\chi$ , 크랩각  $\beta$  의 관계  $\chi = \psi + \beta$  를 설명할 수 있음
- ☐ 두 오차  $x_e$  와  $y_e$  가 각각 무엇을 재고 어디에 쓰이는지 앎
- ☐  $y_e$  가 경로 좌표계의 값이지 동쪽 방향 오차가 아님을 앎
- ☐ LOS 법칙  $\psi_{\text{ref}} = \pi_p - \arctan(y_e/\Delta)$  의 각 항이 무엇인지 앎
- ☐  $\Delta$  를 키우면 무엇이 달라지는지 말할 수 있음
- ☐ 수락반경  $R$  이 가장 짧은 구간보다 커지면 안 되는 이유를 앎
- ☐ 전환 판정 두 가지를 쓰고, 수락 원이 걸릴 수 있는 이유를 설명할 수 있음
- ☐ 헤딩 제어에서 오차를 미분하지 않고  $\dot{r}$  을 되먹임하는 이유를 앎
- ☐ 속도 되먹임에  $u$  가 아니라  $\dot{u}$  를 쓰는 이유를 앎
- ☐ VRX 에 해류가 없다는 것과, 그래서 조류 실험을 어디서 하는지 앎

### 실습 완료

- ☐ `w07_setup` 실행만으로 시뮬레이션이 돌고 창 3개가 떴음
- ☐ 실시간 항적 창에서 배가 경로로 붙는 것을 눈으로 확인했음
- ☐ 오프라인 모델을 LOS 로 실행하고 궤적을 확인했음
- ☐ `guidance_mode = 1` 로 바꿔 atan2 궤적을 확인했음
- ☐ `w07_compare` 로 4조건 표를 얻었음
- ☐ 조류를 커서 크랩각이 커지는 것을 확인했음
- ☐  $\Delta$  를 2 / 3 / 5 / 10 / 20 / 30 으로 바꿔 궤적 변화를 관찰했음
- ☐ VRX 에서 RTF 를 측정했음
- ☐ 페이싱 비율을 RTF 에 맞추고 VRX 모델을 실행했음

- ☐ VRX 에서 배가 경로에 복귀하는 것을 확인했음
- ☐ 주행거리 ÷ 시간 이 **u** 평균과 맞는 것을 확인했음

## 관찰 기록

- ☐ atan2 와 LOS 의 궤적 모양 차이를 그림으로 남겼음
- ☐ 조류 유무에 따른 **mean|y\_e|** 를 표로 남겼음
- ☐ 오프라인과 VRX 궤적을 겹쳐 그려 비교했음

## 과제 7 — 유도법칙 성능 비교

- 제출 기한: 8주차 수업 전
- 제출물: 그래프 + 성능표 + 분석

### ① atan2 vs LOS (조류 유/무)

- w07\_compare** 결과를 표로 제출

| 조건             | 평균  y_e  [m] | 정착 후  y_e  [m] | 완주 시간 [s] | 평균 크랩각 [deg] |
|----------------|--------------|----------------|-----------|--------------|
| atan2 / 조류 없음  |              |                |           |              |
| LOS / 조류 없음    |              |                |           |              |
| atan2 / 조류 0.4 |              |                |           |              |
| LOS / 조류 0.4   |              |                |           |              |

- 궤적 4개를 한 그림에 겹쳐 제출

### ② lookahead distance 스윙

- $\Delta = 2, 3, 5, 10, 20, 30$  m (LOS). 조류 없음과 조류 0.4 m/s(90°) 두 번

#### 조류 없음

| $\Delta$ [m] | 평균  y_e  [m] | 정착 후  y_e  [m] | 완주 시간 [s] | 궤적 특징 |
|--------------|--------------|----------------|-----------|-------|
| 2            |              |                |           |       |
| 3            |              |                |           |       |
| 5            |              |                |           |       |
| 10           |              |                |           |       |
| 20           |              |                |           |       |
| 30           |              |                |           |       |

#### 조류 0.4 m/s (90°)

| $\Delta$ [m] | 평균  y_e  [m] | 정착 후  y_e  [m] | 완주 시간 [s] | 궤적 특징 |
|--------------|--------------|----------------|-----------|-------|
| 2            |              |                |           |       |
| 3            |              |                |           |       |
| 5            |              |                |           |       |
| 10           |              |                |           |       |
| 20           |              |                |           |       |
| 30           |              |                |           |       |

- 각 조건에서 가장 좋은  $\Delta$  를 고르고 근거를 쓸 것
- 두 조건의 최적  $\Delta$  가 같은가 다른가. 다르다면 왜 그런지 쓸 것

### ③ 조류 방향 3종

- 조류 0.4 m/s, 방향 0° / 90° / 180° (LOS) — ① · ② 와 §1-7 표와 같은 세기로 두어 결과를 바로 견줄 수 있게 함
- `current_direction` 은 조류가 흘러가는 방위 (북 = 0°, 시계방향)
- 사각형 미션에서는 한 방향의 조류가 구간마다 역조류 · 옆조류 · 순조류로 바뀜
  - 예: 0° (북쪽으로 흐름) 는 첫 구간(북향)에서는 순조류, 셋째 구간(남향)에서는 역조류
- 평균 크랩각과 평균 |y\_e| 를 측정

| 조류 방향          | 평균 크랩각 [deg] | 평균  y_e  [m] |
|----------------|--------------|--------------|
| 0° (북쪽으로 흐름)   |              |              |
| 90° (동쪽으로 흐름)  |              |              |
| 180° (남쪽으로 흐름) |              |              |

- 표 아래에 각 방향이 어느 구간에서 옆조류인지 적고, 그것으로 결과 차이를 설명할 것

### ④ 오프라인 ↔ VRX 대조

- `w07_vrx_run` 기본값(230 s, 임무 완주)으로 실행 — 오프라인 대조까지 자동으로 함
- 궤적을 겹쳐 그리고 평균 이격 거리를 수치로 제시
- 계인은 바꾸지 말 것

### ⑤ 분석 (8~12줄)

1. 조류가 없을 때도 LOS 가 나은 이유를 `y_e` 를 써서 설명하시오
2. 조류에서 atan2 가 크게 성능이 저하되는 이유를 크랩각으로 설명하시오
3. LOS 도 조류에서 오차가 남음. 왜 그런가?
4. 오프라인 모델과 VRX 가 이만큼 잘 맞는 이유는 무엇인가?

### 평가 기준

| 항목                          | 배점  |
|-----------------------------|-----|
| 네 조건 실행 및 표 완성              | 20% |
| 성능 수치의 정확성 (그래프에서 제대로 읽었는가) | 30% |
| 오프라인 ↔ VRX 대조 실험            | 25% |
| 분석의 타당성                     | 25% |

## 막혔을 때

| 증상                                 | 원인                     | 해결                                                                         |
|------------------------------------|------------------------|----------------------------------------------------------------------------|
| 먼저 <code>w07_setup</code> 을 실행하십시오 | 워크스페이스에 변수 없음          | <code>w07_setup</code> 먼저                                                  |
| 모델은 도는데 배가 안 움직임                   | 페이싱 미설정                | Simulation Pacing 켜기                                                       |
| 주행거리와 <code>u</code> 가 2배 차이       | 페이싱 비율 $\neq$ RTF      | RTF 측정 후 그 값으로 설정                                                          |
| 배가 갑자기 멈춤                          | 육지에 얹힘                 | 웨이포인트를 $x \in [-28, 60]$ , $y \in [-18, 100]$ 안으로                          |
| 배가 마지막 점을 지나 계속 감                  | <code>gate</code> 미연결  | <code>build_w07_models</code> 로 재생성                                        |
| 웨이포인트를 다 건너뛴                       | R 이 웨이포인트 간격보다 큼       | R 을 줄일 것                                                                   |
| 배가 한 바퀴 돌아 목표로 감                   | 각도 wrap 누락             | <code>HeadingErr</code> 의 <code>atan2(sin,cos)</code> 확인                   |
| 헤딩이 반대로 뒀                          | 부호 오류                  | <code>PoseSubscriber</code> 안 <code>Nav</code> 함수의 <code>r = -wz</code> 확인 |
| 코너에서 크게 흔들림                        | $\Delta$ 가 너무 작음       | $\Delta$ 를 키울 것                                                            |
| <code>y_e</code> 가 계속 큼            | <code>atan2</code> 모드임 | <code>guidance_mode = 2</code>                                             |
| MATLAB 에서 토픽이 안 보임                 | Domain ID 불일치          | 6주차 B절 참조                                                                  |

## 참고 자료

### 이 주차의 원본

- Fossen, T. I. *Handbook of Marine Craft Hydrodynamics and Motion Control*, 2nd ed. (2021) §12.3 — 50-원 본 자료 / 에 PDF 있음
- Fossen 2024 Lecture on 2D and 3D path-following control — 50-원 본 자료 / , LOS 유도 전반
- MSS 툴박스 — <https://github.com/cybergalactic/MSS>
  - `SIMULINK/mssSimulinkDemos/` — Otter USV 경로 추종 데모 ( `demoOtterUSVPathFollowingHeadingControl.slx` · `demoOtterUSVPathFollowingCourseControl.slx` )
  - `GNC/crosstrackWpt.m` — 본 과목의 두 오차 식과 같은 회전 형태
  - `GNC/LOSchi.m` — 침로각  $\chi$  지령을 내는 LOS. 전환 판정은 경로 방향( $d - x_e < R_{\text{switch}}$ )
  - `GNC/ILOSpsi.m` · `GNC/ALOSpsi.m` — 선수각  $\psi$  지령을 내는 적분 · 적응 LOS. 지령이 침로나 선수각이냐의 차이는 §1-2
  - 변수 이름 대응은 §1-9 의 표 참고 ( `pi_h` ↔ `pi_p` , `Delta_h` ↔ `Delta` , `R_switch` ↔ `R_LOS` )

## 교수 강의자료 — 이 주차가 그대로 따라간 두 자료

| 자료                                             | 이 주차의 어느 절                                      |
|------------------------------------------------|-------------------------------------------------|
| 2. USV 제 어 기 설 계 .pptx 37~46쪽                  | §1-5 두 오차, §1-6 LOS 법칙, §1-9 수식↔코드 (46쪽이 코드 원본) |
| 2. USV 제 어 기 설 계 .pptx 53~58쪽                  | §1-7 lookahead distance, 가변 $\Delta$            |
| 2. USV 제 어 기 설 계 .pptx 59~71쪽                  | 본 주차 범위 밖 — 이탈량 동역학, ILOS. 대학원 W04 §4-7-4-8     |
| 대학원<br>W04_Waypoint_Following_and_LOS_Guidance | §1-5~§1-8 전부. 유도 과정과 안정성까지 보려면 이쪽               |

### i 기호를 두 자료와 맞춰 두었음

$\pi_p \cdot x_e \cdot y_e \cdot \Delta \cdot R$  은 위 자료들이 쓰는 이름 그대로임  
 대학원 자료는 좌표계를 밝히려고  $x_e^p, y_e^p$  처럼 위첨자를 붙이는데,  
 본 과목은 경로 좌표계 외의 오차를 쓰지 않으므로 위첨자를 생략했음 (§1-9)

## 볼트 내 문서

- [W06\\_Simulink\\_ROS2\\_연동과\\_첫\\_제어기](#) — 내부루프, ground truth, 페이싱
- [W04\\_VRX\\_심화\\_모델구조와\\_토픽조사](#) — 차동 추력배분
- [ENU와-NED를-섞으면-조용히-틀린다](#) — `PoseSubscriber` 안 `Nav` 함수의 좌표 변환
- [조류가-있으면-벃머리와-진행방향이-다르다](#) — 크랩각
- [목표를-향해-가는-것과-경로를-따라가는-것은-다르다](#) — atan2 vs LOS

## 배포 파일

| 파일                                | 내용                                                                                                                                                                                          |
|-----------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| W07_simulink/W07_setup.m          | 학생이 고치는 설정 파일. 실행하면 바로 시뮬레이션이 됨                                                                                                                                                             |
| W07_simulink/W07_animate.m        | 실시간 항적 그리기 (모델 안 <code>Animate</code> 블록이 호출)                                                                                                                                               |
| W07_simulink/W07_0_offline.slx    | 오프라인 WAM-V (조류 실험)                                                                                                                                                                          |
| W07_simulink/W07_1_vrx.slx        | VRX 연동                                                                                                                                                                                      |
| W07_simulink/W07_plot.m           | 성능 그림 5장 + 추진기 그림 2장 + 성능 지표                                                                                                                                                                |
| W07_simulink/W07_sweep.m          | 1-7 · 1-8 절 표 네 개를 다시 재는 스크립트 (오프라인, 1~2 분)                                                                                                                                                 |
| W07_simulink/W07_compare.m        | 4조건 비교 (과제 ①)                                                                                                                                                                               |
| W07_simulink/W07_lookahead_demo.m | $\Delta$ 네 값의 경로 복귀 비교 (§1-7, 실습 D-1)                                                                                                                                                       |
| W07_simulink/W07_crab_demo.m      | 조류 속 벃머리 · 진행 방향 · 크랩각 — 정상 이탈이 $\Delta \tan \beta$ 로 맞는지 확인 (1-7절)                                                                                                                         |
| W07_simulink/W07_vrx_run.m        | VRX 연동 실행 — 토픽 확인 · RTF 측정 · 페이싱 · 오프라인 대조 (실습 E-4)                                                                                                                                         |
| W07_simulink/build_w07_models.m   | 모델 재생성                                                                                                                                                                                      |
| W07_simulink/tidy_layout.m        | 블록 배치 정리                                                                                                                                                                                    |
| W07_matlab/                       | 연구실 참고 코드 (읽기용). <code>LOSchi.m</code> 은 MSS 원본의 연구실 수정본 — 전환 판정이 수락 원. MSS 원본은 <a href="https://github.com/cybergalactic/MSS">https://github.com/cybergalactic/MSS</a> 의 <code>GNC/</code> |

## 다음 주 예고

- 8주차 — 한 점을 중심으로 도는 로이터링
- 웨이포인트로는 원을 제대로 그릴 수 없음. 왜 그런지부터 봄
- 할 일
  - 벡터필드 — 경로를 쫓는 대신 공간 전체에 방향을 깔아 둠
  - 회전 **방향·속도·반경**을 마음대로 바꾸기
  - 반경 오차가 왜 바깥으로 남는지 수치로 분해
- 이번 주차에 작성한 내부루프·추진기·운동모델은 **그대로 씀**. 유도부만 바뀜
- 준비물
  - 과제 7 의 그래프와 표
  - 8주차는 **중간 발표 주간임** — 팀별 Term Project 제안서 초안을 준비할 것

### ★ 중간 발표

평가의 15% 임. 전반부 데모(7주차까지 만든 것)와 Term Project 제안서를 넘  
자세한 내용은 [평가와-팀운영](#) 과 [Term-Project-명세](#) 를 볼 것